OPERATING SYSTEMS

ASSIGNMENT 3 - CO3

Memory Management: Linking, Paging, Segmentation, Virtual Memory, Page Replacement and Disk Management

Name: Mohammed Subahaan H

Register Number: 192572165

Course: Operating System

Course Code: CSA0404

GitHub Repository:
https://github.com/Subahaan-cmd/CSA-0404-OPERATING-SYSTEM

Total Marks: 50

Table of Contents

Table of Contents 2

0. Introduction, Environment and Repository 6

Repository structure 6

Q1 – Linking, Loading and Address Binding (6 Marks) 7

1.1 The problem 7

1.2 From source code to a running process 7

1.3 Address binding 7

1.4 Position-independent code 8

1.5 Implementation: static and dynamic linking 8

1.6 Experimental investigation 12

1.7 Evaluation of static and dynamic linking 14

1.8 Conclusion: which linking method should the bank use? 14

1.9 Alternative concept: loading the library with dlopen() only when needed 14

Q2 – Paging (8 Marks) 15

2.1 The problem 15

2.2 External fragmentation with contiguous allocation 15

2.3 Paging and address translation 16

2.4 Size of the page table 19

2.5 Effective memory-access time 19

2.6 Experimental investigation 20

2.7 Evaluation 21

2.8 Conclusion: the page size for the college server 22

2.9 Alternative concept: an inverted page table instead of a per-process multi-level table . . 22

Q3 – Segmentation (6 Marks) 23

3.1 The problem 23

3.2 Segmentation and address translation 23

3.3 Segmentation with paging (Intel x86) 25

3.4 Segments of a running Linux program 26

3.5 Triggering a segmentation fault . . . 26
3.6 Fragmentation after loading and removing 20 segments . . . 27
3.7 Comparison of the table sizes and of the three schemes . . . 28
3.8 Conclusion: why do modern operating systems rely mainly on paging? . . . 28
3.9 Alternative concept: pure paging instead of segmentation . . . 29
Q4 – Virtual Memory and Demand Paging (8 Marks) 29
4.1 The problem . . . 29
4.2 Handling a page fault . . . 29
4.3 Effective access time for different page-fault rates . . . 30
4.4 Copy-on-write and fast fork . . . 31
4.5 Thrashing and the working-set model . . . 33
4.6 How demand paging lets a 24 GB program run in 16 GB, and the allocation of frames . 33
4.7 Experimental investigation . . . 34
4.8 Conclusion: more RAM or better locality? . . . 36
4.9 Alternative concept: a machine with 32 GB RAM instead of relying on demand paging . 36
Q5 – Page-Replacement Techniques (8 Marks) 37
5.1 The problem . . . 37
5.2 The algorithms and the completed program . . . 37
5.3 Belady's anomaly . . . 39
5.4 Why Optimal is the best but cannot be implemented . . . 40
5.5 Hardware support for LRU and the clock algorithm . . . 40
5.6 Experimental investigation . . . 40
5.7 Engineering decision for the web server . . . 44
5.8 Alternative concept: clock instead of exact LRU . . . 44
Q6 – Disk Management (4 Marks) 45
6.1 The problem . . . 45
6.2 Formatting, partitioning, boot and bad blocks . . . 45
6.3 Partitions and swap on the Linux test system . . . 46
6.4 RAID levels, capacity and probability of data loss . . . 46
6.5 Read speed of SSD and hard disk . . . 48

6.6 Conclusion: recommended storage for the records server 49
6.7 Alternative concept: RAID 6 instead of RAID 5 or RAID 10 49
Q7 – Design a Memory-Management Plan from First Principles (10 Marks) 50
7.1 The problem 50
Stage 1 – Interpret the Problem (2 Marks) 50
Stage 2 – Compare Candidate Mechanisms (3 Marks) 51
Stage 3 – Implement and Verify (3 Marks) 52
Stage 4 – Defend the Design (2 Marks) 56
7.2 Alternative concept: a real-time OS with no virtual memory instead of Linux with locked memory 57
7.3 Evidence summary for Q7 58
8. Conclusion of Assignment 3 58
Appendix A – Additional Source Listings 59
A.1 libenc_v2.c (the library with the security fix, Q1) 59
A.2 q2_edge.c (page-boundary tests, Q2) 59
A.3 q2_calc.py (page-table size, EAT and internal fragmentation, Q2) 59
A.4 q3_frag.py (external fragmentation of 20 segments, Q3) 60
A.5 q4_eat.py (effective access time and frame allocation, Q4) 61
A.6 q5_trace.py (step trace for the hand verification, Q5) 61
A.7 q6_raid.py (RAID capacity and data-loss probabilities, Q6) 62
A.8 sys.sh (system views used in Q6) 63
Appendix B – README.md Template for the GitHub Repository 63
Appendix C – Viva Preparation: Questions and Short Answers 64
Appendix D – Worked Examples for the Viva 65
D.1 Address translation with a page table (page size 1,024, table {5, 2, -1, 7, 0, -1, 3, -1}) . . 65
D.2 Effective access time with a TLB (memory 100 ns, TLB 10 ns) 65
D.3 Page-fault rate for a 10 % slow-down 65
D.4 LRU trace on the textbook string with 3 frames (most recent last) 66
D.5 Capacity of RAID 6 and RAID 10 with six 4 TB disks 66
Appendix E – Test Log 67

Appendix F – Glossary **67**

Appendix G – Extended Calculations **68**

0. Introduction, Environment and Repository

This report answers Assignment 3 of CSA04 – Operating Systems (Course Outcome CO3): linking and loading, paging, segmentation, virtual memory and demand paging, page replacement, disk management, and the design of a memory-management plan. Every question is at Bloom's level K5 (Evaluate). The report follows the progression required by the assignment: **Design, Implementation, Analysis, Evaluation**. All C programs were compiled with `gcc -std=c11 -Wall -Wextra -O2` (or `-std=gnu11` where POSIX or GNU functions such as `mmap` and `dlopen` are used) with gcc 13.3.0 on Ubuntu 24.04 (Linux 6.18) and executed. The outputs printed in this report are the actual outputs of those runs. Python 3.12 was used for the calculations and for the graphs.

> **Limitations of the test machine.** The machine is a virtual machine with 1 vCPU, 4 GB of RAM, **no swap space** and a single virtual SSD (no hard disk). Therefore: (1) the measurement of run time as memory grows beyond RAM could not be done on the real machine (it would invoke the out-of-memory killer, as no swap exists); a model and simulation is used instead and a procedure for a VM with a memory limit is given. (2) The SSD against hard-disk read-speed comparison is only possible for the SSD. (3) Timing values are noisy. The student should repeat these on a personal machine and add the screenshots to the repository.

Repository structure

```
CSA04-<RegisterNumber>-Assignment3/
  README.md  (name, register number, gcc 13.3.0, Ubuntu 24.04, compile commands, run \
       steps, result of each question)
  Q1/ libenc.c libenc_v2.c app.c app_dl.c demo.sh    outputs/
  Q2/ q2_page.c q2_edge.c q2_faults.c q2_calc.py     outputs/
  Q3/ q3_seg.c q3_segv.c q3_frag.py maps.sh          outputs/
  Q4/ q4_touch.c q4_cow.c q4_eat.py                  outputs/
  Q5/ q5_replace.c q5_exp.c plot.py faults.csv       outputs/ fig_faults.png
  Q6/ q6_raid.py sys.sh                              outputs/
  Q7/ q7_mlock.c q7_sim.py                           outputs/
```

At least one commit is made for each question. The name, register number and repository link on the cover page are placeholders for the student to complete.

Q1 – Linking, Loading and Address Binding (6 Marks)

1.1 The problem

A bank's branch software is built from many modules and libraries. When a security fix is made to the encryption library, every branch currently receives a new 200 MB executable. The question is how the program should be linked so that the fix can be distributed cheaply and safely.

1.2 From source code to a running process

Figure 1.1 From source to a running process: compile, link, load

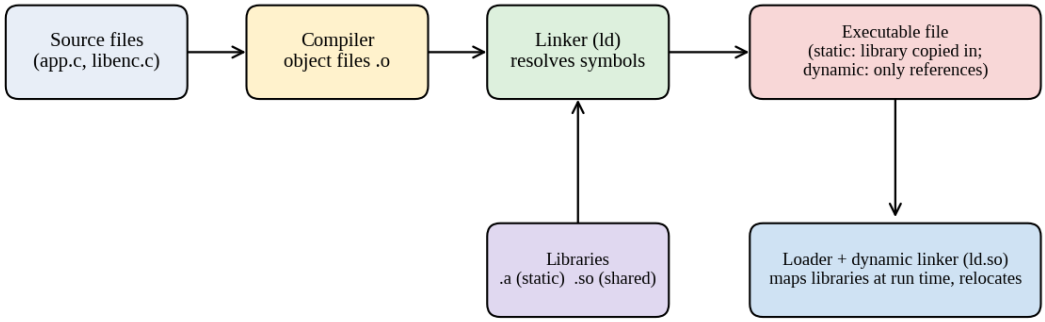


Figure 1.1 Compile, link and load

The **compiler** translates each source file into an object file that contains machine code, a symbol table and *relocation information* (places where addresses must be filled in). The **linker** combines object files and libraries into one executable: it resolves symbols (a call to `encrypt` is matched to its definition), assigns the relative addresses of sections, and applies relocations. With *static linking* the code of the library is copied into the executable. With *dynamic linking* the executable contains only the name of the library and a table of references; the work is completed at load time. The **loader** (the kernel's `execve` plus the dynamic linker `ld.so`) reads the executable, creates the process's address space, maps the code and data segments, maps the shared libraries into the address space and performs the remaining relocations, and then starts the program.

1.3 Address binding

Instructions and data names in the source code are *symbolic*; they must be bound to memory addresses. Binding can happen at three times.

- **Compile time.** If the position of the process in memory is known in advance, the compiler generates *absolute code* (the MS-DOS `.COM` format). If the starting location

changes, the program must be recompiled. This is used in small embedded systems.

- **Load time.** The compiler generates *relocatable code* (addresses relative to the start of the module). The loader fixes the final addresses when the program is loaded. If the starting address changes, the program only needs to be reloaded.
- **Execution time.** The binding is delayed until run time, and the process can be moved in memory during execution. This needs hardware support: the MMU translates each logical address to a physical address on every access (base-limit registers, or paging). This is the method used by all modern general-purpose operating systems and gives each process a private logical address space.

1.4 Position-independent code

Shared libraries are mapped at different addresses in different processes (and, with address-space layout randomisation, at a different address in every run). The code must therefore work wherever it is mapped. **Position-independent code (PIC)**, compiled with `-fPIC`, uses only *relative* addressing for jumps and for access to data (program-counter-relative addressing) and reaches global data and external functions through the *global offset table (GOT)* and the *procedure linkage table (PLT)*, whose entries are filled in by the dynamic linker. Because the code itself is never modified by relocation, its pages are read-only and can be **shared by all processes** that map the library; only the small GOT and data pages are private. Non-PIC code would need to be patched for each load address, and then every process would have its own private modified copy.

1.5 Implementation: static and dynamic linking

The encryption library (`libenc.c`, version 1) exports `enc_version()` and `encrypt()`. Version 2 (`libenc_v2.c`) is the "security fix" with a stronger key schedule. The application `app.c` encrypts a record, prints the library version and its own resident memory, and can print its memory map.

Listing: libenc.c

```
/* The bank's encryption library, version 1 (simple XOR cipher, for demonstration on \
      ly). */
#include <stddef.h>
int enc_version(void) { return 1; }
void encrypt(char *buf, size_t n) { for (size_t i = 0; i < n; i++) buf[i] ^= 0x5A; }
```

Listing: app.c

```
#define _DEFAULT_SOURCE
#include <unistd.h>
/* Branch application: encrypts a record with the library and reports memory use. */
#include <stdio.h>
#include <string.h>
```

```
#include <stdlib.h>
size_t strlen(const char *);
int enc_version(void);
void encrypt(char *buf, size_t n);
static long rss_kb(void) { FILE *f = fopen("/proc/self/status", "r"); char l[128]; l \
      ong v = 0; while (fgets(l, sizeof l, f)) if (!strncmp(l, "VmRSS:", 6)) { v = a \
      tol(l + 6); break; } fclose(f); return v; }
int main(int argc, char **argv)
{
    char rec[] = "PATIENT-4711:HbA1c=6.1";
    encrypt(rec, strlen(rec));
    printf("library version %d, ciphertext:", enc_version());
    for (size_t i = 0; i < 8; i++) printf(" %02x", (unsigned char)rec[i]);
    printf("\nresident memory: %ld KB\n", rss_kb());
    if (argc > 1 && !strcmp(argv[1], "--maps")) {
        FILE *f = fopen("/proc/self/maps", "r"); char l[256];
        while (fgets(l, sizeof l, f)) if (strstr(l, "app") || strstr(l, "libenc") || \
       strstr(l, "libc") || strstr(l, "[stack]") || strstr(l, "[heap]") || strstr(l, \
       "ld-linux")) fputs(l, stdout);
        fclose(f);
    }
    if (argc > 1 && !strcmp(argv[1], "--wait")) { sleep(3); }
    return 0;
}
```

Listing: app_dl.c

```
/* Run-time loading with dlopen()/dlsym(): the library is loaded only when encryptio \
      n is needed. */
#include <stdio.h>
#include <string.h>
#include <dlfcn.h>
int main(int argc, char **argv)
{
    int need = argc > 1 && !strcmp(argv[1], "encrypt");
    printf("start: encryption %s needed\n", need ? "is" : "is NOT");
    if (!need) return 0;
    void *h = dlopen("./libenc.so", RTLD_NOW);
    if (!h) { fprintf(stderr, "cannot load library: %s\n(falling back: records are s \
      ent over TLS only)\n", dlerror()); return 2; }
    void (*enc)(char *, size_t) = (void (*)(char *, size_t))dlsym(h, "encrypt");
    int (*ver)(void) = (int (*)(void))dlsym(h, "enc_version");
    if (!enc || !ver) { fprintf(stderr, "symbol missing: %s\n", dlerror()); return 3 \
      ; }
    char rec[] = "PATIENT-4711";
    enc(rec, strlen(rec));
    printf("loaded library version %d at run time; first byte now %02x\n", ver(), (u \
      nsigned char)rec[0]);
    dlclose(h);
    return 0;
}
```

The libraries were built with `gcc -fPIC -shared -o libenc.so libenc.c`, and the program in two ways. All the experiments of this question were run by the script `demo.sh`; its output is shown below and explained in the following sections.

Listing: demo.sh

```
#!/usr/bin/env bash
# Q1 experiments: static v dynamic linking, replacing the library, missing library,  \
      start-up time, shared pages
cd "$(dirname "$0")"
run() { echo "\$ $*"; eval "$*" 2>&1; }
run "gcc -std=gnu11 -O2 -static -o app_static app.c libenc.c"
run "gcc -std=gnu11 -O2 -o app_dynamic app.c -L. -lenc -Wl,-rpath,'\$ORIGIN'"
run "ls -l app_static app_dynamic libenc.so | awk '{print \$5, \$9}'"
run "size app_static app_dynamic"
run "ldd app_dynamic"
run "ldd app_static"
echo; echo "--- replace the shared library WITHOUT recompiling (security fix = versi \
      on 2)"
run "./app_dynamic | head -1"; run "./app_static | head -1"
cp libenc.so /tmp/libenc_v1.so; cp libenc_v2.so libenc.so
run "./app_dynamic | head -1   # dynamic program now uses the fixed library"
run "./app_static | head -1    # static program still contains the old code"
cp /tmp/libenc_v1.so libenc.so
echo; echo "--- library missing"
mv libenc.so /tmp/hold.so
run "./app_dynamic; echo exit status=\$?"
run "./app_dl encrypt; echo exit status=\$?"
mv /tmp/hold.so libenc.so
echo; echo "--- start-up time: 500 runs each"
s=$(date +%s.%N); for i in $(seq 1 500); do ./app_dynamic > /dev/null; done; m=$(dat \
      e +%s.%N); for i in $(seq 1 500); do ./app_static > /dev/null; done; e=$(date  \
      +%s.%N)
echo "dynamic: $(echo "($m-$s)*2" | bc) ms per run; static: $(echo "($e-$m)*2" | bc) \
       ms per run"
run "./app_dynamic | tail -1; ./app_static | tail -1"
echo; echo "--- two copies running: which pages are shared (smaps_rollup)"
./app_dynamic --wait > /dev/null & sleep 0.4; ./app_dynamic --wait > /dev/null & sle \
      ep 0.4
for p in $(pgrep app_dynamic); do echo "PID $p: $(grep -E '^(Rss|Pss|Shared_Clean|Pr \
      ivate_Dirty)' /proc/$p/smaps_rollup | tr -s ' ' | tr '\n' ' ')"; done; wait
```

`$ bash q1/demo.sh`

```
$ gcc -std=gnu11 -O2 -static -o app_static app.c libenc.c
$ gcc -std=gnu11 -O2 -o app_dynamic app.c -L. -lenc -Wl,-rpath,'$ORIGIN'
$ ls -l app_static app_dynamic libenc.so | awk '{print $5, $9}'
16472 app_dynamic
785704 app_static
15152 libenc.so
$ size app_static app_dynamic
   text       data        bss        dec        hex    filename
```

```
 670266      23376      22472     716114      aed52    app_static
   3308        720         16       4044        fcc    app_dynamic
$ ldd app_dynamic
    linux-vdso.so.1 (0x00007ff9a8656000)
    libenc.so => /home/claude/work/a3/q1/./libenc.so (0x00007ff9a8644000)
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007ff9a8400000)
    /lib64/ld-linux-x86-64.so.2 (0x00007ff9a8658000)
$ ldd app_static
    not a dynamic executable

--- replace the shared library WITHOUT recompiling (security fix = version 2)
$ ./app_dynamic | head -1
library version 1, ciphertext: 0a 1b 0e 13 1f 14 0e 77
$ ./app_static | head -1
library version 1, ciphertext: 0a 1b 0e 13 1f 14 0e 77
$ ./app_dynamic | head -1   # dynamic program now uses the fixed library
library version 2, ciphertext: f7 87 b1 4d 66 0c 35 ad
$ ./app_static | head -1    # static program still contains the old code
library version 1, ciphertext: 0a 1b 0e 13 1f 14 0e 77

--- library missing
$ ./app_dynamic; echo exit status=$?
./app_dynamic: error while loading shared libraries: libenc.so: cannot open shared o \
      bject file: No such file or directory
exit status=127
$ ./app_dl encrypt; echo exit status=$?
cannot load library: ./libenc.so: cannot open shared object file: No such file or di \
      rectory
(falling back: records are sent over TLS only)
start: encryption is needed
exit status=2

--- start-up time: 500 runs each
dynamic: 1.096599774 ms per run; static: .852881494 ms per run
$ ./app_dynamic | tail -1; ./app_static | tail -1
resident memory: 1612 KB
resident memory: 724 KB

--- two copies running: which pages are shared (smaps_rollup)
PID 5002: Rss: 1852 kB Pss: 304 kB Pss_Dirty: 118 kB Pss_Anon: 112 kB Pss_File: 192  \
      kB Pss_Shmem: 0 kB Shared_Clean: 1728 kB Private_Dirty: 112 kB
PID 5004: Rss: 1856 kB Pss: 324 kB Pss_Dirty: 122 kB Pss_Anon: 116 kB Pss_File: 208  \
      kB Pss_Shmem: 0 kB Shared_Clean: 1728 kB Private_Dirty: 116 kB
```

Sizes and library lists

The statically linked executable is **785,704 bytes** (the whole C library is copied in: 670,266 bytes of code) while the dynamically linked executable is only **16,472 bytes** (code 3,308 bytes) plus the shared library file `libenc.so` of 15,152 bytes. The `ldd` command lists the shared libraries that the dynamic program needs: `linux-vdso.so.1` (a kernel-provided page that makes some system calls fast), `libenc.so` (the bank's library, found through the

run path), `libc.so.6` (the C library) and `ld-linux-x86-64.so.2` (the dynamic linker). For the static program `ldd` answers "not a dynamic executable". For the real bank, with a 200 MB executable the effect would be larger: if the encryption library is 2 MB, then a security fix with static linking means shipping 200 MB to every branch, whereas with a shared library only the 2 MB library is replaced.

Addresses at which the program and the libraries are loaded

The program prints its own memory map (`/proc/self/maps`) with the option `--maps`:

```
$ cd q1 && ./app_dynamic --maps | head -14
```

```
library version 1, ciphertext: 0a 1b 0e 13 1f 14 0e 77
resident memory: 1600 KB
55ddee316000-55ddee317000 r--p 00000000 fe:00 573634                     /home/claud \
      e/work/a3/q1/app_dynamic
55ddee317000-55ddee318000 r-xp 00001000 fe:00 573634                     /home/claud \
      e/work/a3/q1/app_dynamic
55ddee318000-55ddee319000 r--p 00002000 fe:00 573634                     /home/claud \
      e/work/a3/q1/app_dynamic
55ddee319000-55ddee31a000 r--p 00002000 fe:00 573634                     /home/claud \
      e/work/a3/q1/app_dynamic
55ddee31a000-55ddee31b000 rw-p 00003000 fe:00 573634                     /home/claud \
      e/work/a3/q1/app_dynamic
55ddf295c000-55ddf297d000 rw-p 00000000 00:00 0                          [heap]
7f8ef9c00000-7f8ef9c28000 r--p 00000000 fe:00 191494                     /usr/lib/x8 \
      6_64-linux-gnu/libc.so.6
7f8ef9c28000-7f8ef9db0000 r-xp 00028000 fe:00 191494                     /usr/lib/x8 \
      6_64-linux-gnu/libc.so.6
7f8ef9db0000-7f8ef9dff000 r--p 001b0000 fe:00 191494                     /usr/lib/x8 \
      6_64-linux-gnu/libc.so.6
7f8ef9dff000-7f8ef9e03000 r--p 001fe000 fe:00 191494                     /usr/lib/x8 \
      6_64-linux-gnu/libc.so.6
7f8ef9e03000-7f8ef9e05000 rw-p 00202000 fe:00 191494                     /usr/lib/x8 \
      6_64-linux-gnu/libc.so.6
7f8ef9f21000-7f8ef9f22000 r--p 00000000 fe:00 573632                     /home/claud \
      e/work/a3/q1/libenc.so
```

The map shows the executable mapped at `0x55...` (a randomised address, because it is a position-independent executable), the heap, `libc.so.6` at `0x7f...` in four parts (read-only header, **r-xp** executable code, read-only data, and **rw-p** writable data), `libenc.so` in the same four-part pattern, and the dynamic linker `ld-linux`. The addresses change on every run (ASLR), which proves that the code is position independent. The permission pattern shows the segmentation of the process: code is executable and read-only, data is writable but not executable.

1.6 Experimental investigation

Start-up time and memory use

In 500 consecutive runs, the dynamically linked version took 1.10 ms per run and the static one 0.85 ms per run (about 29 % slower to start): the dynamic linker must open and map `libenc.so` and `libc.so`, look up symbols and apply relocations. The resident memory of one process was 1,612 KB for the dynamic program and 724 KB for the static one: for a single process the dynamic version looks *larger* because the shared libraries' pages are counted in its resident set (the size of the whole mapped libc pages touched during loading). The measurement for two running copies below shows the real picture.

Replace the library without recompiling

The shared library file was replaced by version 2 (`cp libenc_v2.so libenc.so`). The dynamic program, without recompilation, printed *library version 2* and a different ciphertext (`f7 87 b1 4d ...` instead of `0a 1b 0e 13 ...`); the static program still printed version 1 because the old code is inside its executable. This is the central advantage of dynamic linking for the bank: the security fix is deployed by replacing a file, with no new executable. The same property is a risk: an incompatible library version can break all programs that use it ("dependency hell"), and a malicious library placed earlier in the search path would be loaded by every program (the reason for `LD_LIBRARY_PATH` restrictions and signed packages).

Two copies running: which pages are shared

Two copies of the dynamic program were started at the same time and `/proc/<pid>/smaps_rollup` was read. Each shows a resident size (Rss) of about 1,850 KB but a *proportional* size (Pss) of only 304 to 324 KB. The Pss counts each shared page divided by the number of processes sharing it. Out of the resident 1,852 KB, **1,728 KB is Shared_Clean** (read-only code and data of the libraries and the executable, the same physical pages in both processes) and only about 112 to 116 KB is Private_Dirty (the heap, stack and GOT/data pages that each process modified). With a static executable each copy would have its own copy of the library code in its (shared) text segment, so the two copies of the *same* static program also share their text; but two *different* static programs that use the same library would each carry a separate copy in memory. With dynamic libraries, all hundreds of different programs share a single physical copy of libc.

Missing library

When `libenc.so` was removed, the dynamically linked program refused to start: the loader printed *error while loading shared libraries: libenc.so: cannot open shared object file* and the exit status was 127. The program that uses `dlopen` loaded nothing at start, so it could report the problem itself and continue (*falling back to TLS only*), with exit status 2 chosen by the programmer. The static program is immune to this failure because it contains everything it needs.

1.7 Evaluation of static and dynamic linking

Criterion	Static linking	Dynamic (shared) linking	Measured evidence
Executable size	Large (libraries copied in)	Small	785,704 against 16,472 bytes
Start-up time	Faster: no library search, no run-time relocation	Slower: loader maps and relocates libraries	0.85 ms against 1.10 ms
Memory with many processes	Each program has its own copy of library code	One shared physical copy, only data pages are private	1,728 KB of the 1,852 KB resident is shared
Updating a library (security fix)	Every program must be rebuilt and redistributed	Replace one file	Version 2 picked up without recompiling
Dependency problems	None: self-contained	Missing or incompatible library breaks the program	Exit status 127 when missing
Security	Vulnerable code stays inside old binaries until rebuilt	Fix applies to all programs at once; but library hijacking is possible	
Portability	Runs on any machine of the same architecture	Needs compatible libraries installed	

1.8 Conclusion: which linking method should the bank use?

The bank should use **dynamic linking** for its branch software, with the encryption library as a versioned shared library delivered through a signed package, and with the application built so that it checks the library version at start-up. Reasons: (1) a security fix must be distributed quickly; replacing a 15 KB library is far cheaper and faster than shipping a 200 MB executable to each branch; (2) memory is shared between the programs on a branch server (thousands of processes of tellers' sessions share one copy of the library code); (3) the start-up penalty (0.25 ms) is irrelevant for a banking application. The risks (missing or incompatible library, hijacking) are handled by version checks, a fixed library path (an `rpath` or a system directory writable only by root), signed packages, and a staged roll-out with a rollback. Static linking is retained only for small emergency tools that must run on a damaged system.

1.9 Alternative concept: loading the library with dlopen() only when needed

What the solution would be. The program does not list the library at link time. When encryption is needed it calls `dlopen("./libenc.so", RTLD_NOW)` and obtains the functions with `dlsym`, as in `app_dl.c`.

Start-up time. The program starts without loading the library: in the run `./app_dl` (no encryption needed) the program printed "encryption is NOT needed" and exited without ever touching the library. The start-up work is O(1) in the number of optional libraries (nothing is loaded), instead of O(L) for L libraries linked at start-up; the cost moves to the first use, where `dlopen` adds a delay (opening, mapping and relocating: of the order of 100 µs to 1 ms) at the moment the first record is encrypted, which may be unwanted for a latency-sensitive transaction (it can be hidden by loading in a background thread).

Memory used when encryption is not needed. With start-up linking the library is mapped and (partly) resident in every process, even if encryption is never used; with `dlopen` the memory is zero until used (the 15 KB library is small, but a real cryptographic library of several MB, multiplied by thousands of processes, is considerable, although the code pages are shared).

Error handling. With link-time dynamic linking, a missing library prevents the program from starting at all (exit status 127, no chance to react). With `dlopen` the program handles the failure explicitly: `dlopen` returns NULL, `dlerror()` gives the message, and the program can fall back, report, or retry. The cost is more code: every `dlsym` result must be checked, the function pointer types are not checked by the compiler, and the programmer must avoid calling the library when the load failed (the run printed the fallback message and exit status 2). The recommended design for the bank is link-time dynamic linking for the core encryption (always needed) and `dlopen` for optional modules (plug-ins, optional report formats).

Q2 – Paging (8 Marks)

2.1 The problem

A college server runs hundreds of student programs of very different sizes. Contiguous allocation left many small unusable gaps, and large programs could not be loaded although enough total memory was free.

2.2 External fragmentation with contiguous allocation

Figure 2.2 External fragmentation: 60 MB free in three holes (20, 15, 25) but a 40 MB program does not fit

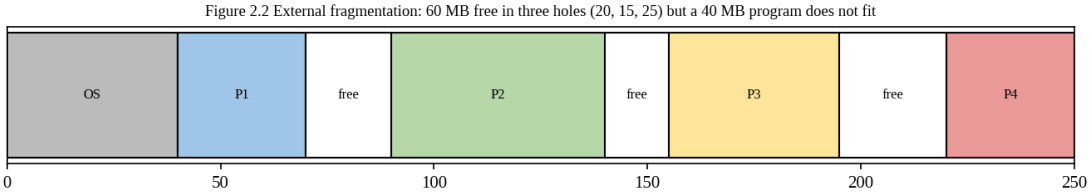


Figure 2.2 External fragmentation

With contiguous allocation each process gets one block of memory. As processes start and terminate, free memory is broken into many holes. In the diagram the free memory is 20 + 15 + 25 = 60 MB, but the largest hole is only 25 MB, so a 40 MB program cannot be loaded: this is **external fragmentation** (free memory exists, but not contiguously). The first-fit rule leaves, on average, one-third of memory unusable (the "50-percent rule": for N allocated blocks about N/2 blocks are lost to fragmentation). **Compaction** can merge the holes but is expensive (copying all processes) and only possible with execution-time binding.

2.3 Paging and address translation

Figure 2.1 Address translation with a page table (page size 1 KB)

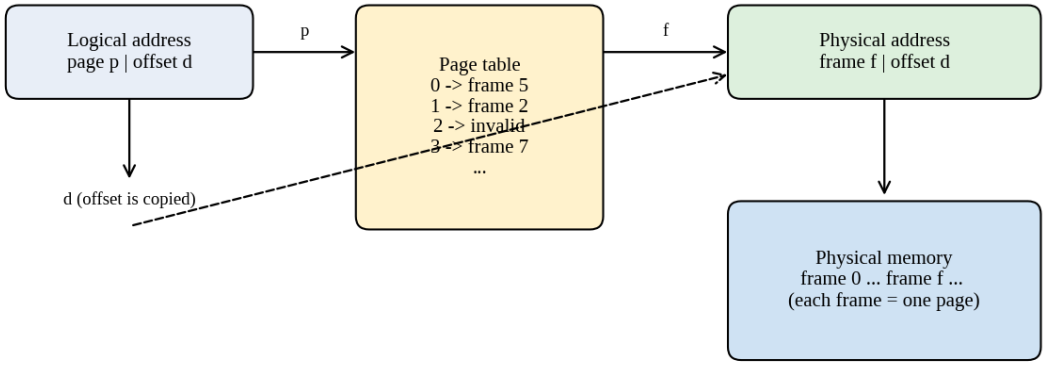


Figure 2.1 Translation with a page table

Paging divides the logical address space into fixed-size **pages** and physical memory into **frames** of the same size. A logical address is split into a page number p (the high-order bits) and an offset d (the low-order bits); with page size 1,024 bytes, p = address / 1024 and d = address mod 1024. The page table gives the frame number f for page p, and the physical address is f × 1024 + d. Any free frame can hold any page, so the pages of a process need not be contiguous, which removes external fragmentation.

Completed program

TODO 1 was replaced by the split `p = a / PAGE_SIZE`, `d = a % PAGE_SIZE`, a check that the page number is inside the table (otherwise "invalid address"), a check of the frame number (-1 means the page is not in memory: "page fault") and the calculation of the physical address. The page size is a macro with default 1,024 so that the 2,048 case can be compiled with `-DPAGE_SIZE_VAL=2048` without editing the code.

Listing: q2_page.c

```
#include <stdio.h>
#ifndef PAGE_SIZE_VAL
#define PAGE_SIZE_VAL 1024
#endif
#define PAGE_SIZE PAGE_SIZE_VAL
#define PAGES 8

/* Page table of a process: frame number, or -1 if the page is not in memory. */
int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};

int main(void)
{
    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};
    for (int i = 0; i < 6; i++) {
        unsigned a = logical[i];
        /* TODO 1 (completed) */
        unsigned p = a / PAGE_SIZE, d = a % PAGE_SIZE;
        printf("%u -> page %u offset %u : ", a, p, d);
        if (p >= PAGES) printf("invalid address\n");
        else if (page_table[p] == -1) printf("page fault\n");
        else printf("physical %u\n", (unsigned)page_table[p] * PAGE_SIZE + d);
    }
    return 0;
}
```

$ cd q2 && gcc -std=gnu11 -Wall -Wextra -o q2_page q2_page.c && ./q2_page

```
0 -> page 0 offset 0 : physical 5120
1500 -> page 1 offset 476 : physical 2524
3100 -> page 3 offset 28 : physical 7196
6200 -> page 6 offset 56 : physical 3128
9000 -> page 8 offset 808 : invalid address
2049 -> page 2 offset 1 : page fault
```

The output is identical to the expected output. Examples: address 1,500 is page 1, offset 476; page 1 is in frame 2, so the physical address is 2 × 1,024 + 476 = 2,524. Address 9,000 is page 8, which is outside the table of 8 pages (0 to 7), so the address is invalid (the hardware raises a trap). Address 2,049 is page 2 offset 1; page 2 has frame -1, so a page fault occurs.

Page size 2,048: hand calculation and verification

With 2,048-byte pages (the table is unchanged and has 8 entries, so the address space is now 16 KB):

Logical address	p = a / 2048	d = a mod 2048	Frame	Physical address (hand)	Program output
0	0	0	5	5 x 2048 + 0 = 10,240	10240
1500	0	1500	5	10,240 + 1,500 = 11,740	11740
3100	1	1052	2	2 x 2048 + 1052 = 5,148	5148
6200	3	56	7	7 x 2048 + 56 = 14,392	14392
9000	4	808	0	0 x 2048 + 808 = 808	808
2049	1	1	2	4,096 + 1 = 4,097	4097

```
$ cd q2 && ./q2_page2048
```

```
0 -> page 0 offset 0 : physical 10240
1500 -> page 0 offset 1500 : physical 11740
3100 -> page 1 offset 1052 : physical 5148
6200 -> page 3 offset 56 : physical 14392
9000 -> page 4 offset 808 : physical 808
2049 -> page 1 offset 1 : physical 4097
```

The program output agrees with the hand calculation. The same addresses now give different results (9,000 is valid, 2,049 no longer faults), because a larger page maps a larger region with the same table entry: the page-size choice changes both the address-space coverage and the fragmentation.

Addresses at the start and end of a page and beyond the last page

```
$ cd q2 && ./q2_edge
```

```
address     0: page 0 offset    0 -> physical 5120 (frame 5)
address  1023: page 0 offset 1023 -> physical 6143 (frame 5)
address  1024: page 1 offset    0 -> physical 2048 (frame 2)
address  2047: page 1 offset 1023 -> physical 3071 (frame 2)
address  2048: page 2 offset    0 -> page fault (valid page, not in memory)
address  7167: page 6 offset 1023 -> physical 4095 (frame 3)
address  8191: page 7 offset 1023 -> page fault (valid page, not in memory)
address  8192: page 8 offset    0 -> INVALID (beyond the last page: trap, process ki \
      lled)
address  8193: page 8 offset    1 -> INVALID (beyond the last page: trap, process ki \
      lled)
```

An offset of 0 is the first byte of a page and an offset of 1,023 the last byte; the next address (1,024) belongs to the next page, with a different frame. Pages 2, 5 and 7 are valid but not

in memory (page fault, handled by the OS by loading the page), and the addresses 8,192 and above are beyond the last page: they are illegal and the process would be terminated.

2.4 Size of the page table

For a 32-bit address space with 4 KB pages (12 offset bits), the number of pages is 2^20 = 1,048,576; with 4-byte entries the page table is 4 MB per process. For a 48-bit address space the number of pages is 2^36 = 68,719,476,736; with 8-byte entries a single-level table would be 512 GB per process, which is impossible, hence the need for **hierarchical (multi-level) page tables**: x86-64 uses four levels of 512 entries (9 bits each, 4 × 9 + 12 = 48 bits). Only the parts of the tree that are in use exist, so a small process needs one page for each of the four levels plus the leaf tables for its mapped regions: for 10 MB of contiguous memory about 6 pages = 24 KB.

2.5 Effective memory-access time

Without a TLB every logical memory access needs one access to the page table in memory and one to the data: 2 × 100 ns = 200 ns for a one-level table, and 5 × 100 = 500 ns for four levels. The **translation look-aside buffer (TLB)** is a small, fast associative cache of recent translations. If the TLB lookup takes 10 ns, then with hit ratio h: EAT = h × (10 + 100) + (1 - h) × (10 + 100 + 100) ns for a one-level table.

```
$ cd q2 && python3 q2_calc.py
```

```
== Page-table size (single level, 4-byte entries for 32-bit, 8-byte entries for 48-b \
      it), page size 4 KB
  32-bit address space: 1,048,576 entries x 4 bytes = 4 MB
  48-bit address space: 68,719,476,736 entries x 8 bytes = 524288 MB
  (48-bit: 4-level table: only used parts exist; a 10 MB process needs about 1 + 1 + \
       1 + 3 = 6 pages = 24 KB)

== Effective memory-access time, memory 100 ns, TLB 10 ns, 1-level table
  without TLB (1 level): 200 ns; (4 levels): 500 ns
  hit ratio 80%: 1 level 130.0 ns  (slow-down 30% over 100 ns); 4 levels 190.0 ns
  hit ratio 95%: 1 level 115.0 ns  (slow-down 15% over 100 ns); 4 levels 130.0 ns
  hit ratio 99%: 1 level 111.0 ns  (slow-down 11% over 100 ns); 4 levels 114.0 ns

== Internal fragmentation for 100 processes with random sizes (seed 42, sizes 10 KB  \
      - 5 MB; 2 MB pages use 1-64 MB processes)
  page  1 KB: total memory requested     229.9 MB, internal fragmentation       47.0 \
       KB (0.0200%), average 0.5 KB per process, pages needed 235417
  page  4 KB: total memory requested     229.9 MB, internal fragmentation      190.0 \
       KB (0.0807%), average 1.9 KB per process, pages needed 58890
  page  2 MB: total memory requested    3359.4 MB, internal fragmentation   111252.3 \
       KB (3.2341%), average 1112.5 KB per process, pages needed 1734
  expected waste per process = page size / 2: 0.5 KB, 2 KB, 1024 KB
```

For hit ratios of 80 %, 95 % and 99% the effective access time is 130, 115 and 111 ns (30 %, 15 % and 11 % slower than direct memory); with four-level tables 190, 130 and 114 ns. A high hit ratio matters much more for deep page tables: with a 4-level table a miss costs 5 memory references. Programs with locality reach 99 % or more.

2.6 Experimental investigation

Internal fragmentation for 100 processes

For 100 random process sizes (seed 42, between 10 KB and 5 MB), the total internal fragmentation was 47.0 KB for 1 KB pages (average 0.5 KB per process), 190.0 KB for 4 KB pages (1.9 KB per process), and, for 100 processes with sizes between 1 and 64 MB (3,359 MB in all), 111,252 KB (about 109 MB, average 1,112 KB per process, 3.2 % of the memory) with 2 MB pages. The expected waste per process is half a page on average (0.5 KB, 2 KB, 1,024 KB), which agrees with the measurements.

Page size and page faults on Linux

`getconf PAGESIZE` reports 4,096 bytes. The program `q2_faults.c` touches 25,600 pages (100 MB) and counts page faults with `getrusage`:

Listing: q2_faults.c

```
/* Count the page faults of a program: touch N pages of fresh memory and read the mi \
      nor-fault counter. */
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/resource.h>
static long minflt(void) { struct rusage r; getrusage(RUSAGE_SELF, &r); return r.ru_ \
      minflt; }
int main(void)
{
    long ps = sysconf(_SC_PAGESIZE);
    size_t pages = 25600;                          /* 100 MB with 4 KB pages */
    char *m = mmap(NULL, pages * ps, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONY \
      MOUS, -1, 0);
    long f0 = minflt();
    for (size_t i = 0; i < pages; i++) m[i * ps] = 1;
    long f1 = minflt();
    for (size_t i = 0; i < pages; i++) m[i * ps] = 2;
    long f2 = minflt();
    printf("page size = %ld bytes\nfirst pass over %zu pages : %ld page faults\nseco \
      nd pass (pages now mapped): %ld page faults\n", ps, pages, f1 - f0, f2 - f1);
    return 0;
}
```

```
$ cd q2 && ./q2_faults
```

```
page size = 4096 bytes
first pass over 25600 pages : 25600 page faults
second pass (pages now mapped): 0 page faults
```

The first touch of every page caused one page fault (25,600 faults for 25,600 pages) and the second pass caused none: the page-table entries now exist. (This program uses the `getrusage` call because `/usr/bin/time -v` was not installed in the test environment; `/usr/bin/time -v ./program` prints the same "Minor page faults" counter, and it should be run on the student's machine for the screenshot.)

2.7 Evaluation

How paging removes external fragmentation, and why internal fragmentation remains. Every frame is the same size, so any free frame fits any page, and there are no unusable holes: a process can be loaded whenever the *number* of free frames is sufficient. But the last page of a process is rarely full: on average half a page is wasted in each process (internal fragmentation), as the experiment showed.

Page sizes of 1 KB, 4 KB and 2 MB.

Page size	Internal fragmentation per process (average)	Page-table size for a 1 GB process (4-byte entries)	TLB coverage with 64 entries	Disk transfer efficiency	Verdict
1 KB	0.5 KB	4 MB (1,048,576 entries)	64 KB	Poor: seek time dominates small transfers	Too small: large tables, poor TLB coverage
4 KB	2 KB	1 MB (262,144 entries)	256 KB	Good compromise	Standard choice
2 MB (huge page)	1 MB	2 KB (512 entries)	128 MB	Best for sequential access, but a large page is expensive to read and write on a page fault	Good for big-memory databases; wasteful for small processes

Hierarchical, hashed and inverted page tables. A *hierarchical* table splits the page number into several parts (for example 10 + 10 bits) so that unused parts of the address space need no table: space is proportional to the memory actually used, and a translation walks several levels (a TLB miss costs one reference per level). A *hashed* page table (used for address spaces larger than 32 bits) hashes the page number into a table of chains: a lookup is O(1) on average, each entry holds the virtual page number, the frame and a

pointer to the next element. An *inverted* page table has one entry per physical frame (page number and process ID): its size is proportional to physical memory, independent of the number of processes, but a lookup needs a hash search (a linear search without hashing), and shared pages are harder to implement.

Protection and shared pages. Every page-table entry has a *valid–invalid bit* (the page belongs to the process's address space or not), *protection bits* (read, write, execute, user/supervisor), and often dirty and accessed bits. An access that violates the permission traps to the OS. Pages are shared simply by letting the page tables of several processes point to the same frame: re-entrant code (the shared library of Q1, the compiler used by all students) exists once in memory and is mapped read-only into every process.

2.8 Conclusion: the page size for the college server

A page size of **4 KB** is chosen for general student processes: it keeps the internal fragmentation at about 2 KB per process (0.08 % in the experiment), has a page-table size that the multi-level design handles well, and a TLB with 64 to 1,536 entries covers 256 KB to 6 MB. With hundreds of student programs of varying sizes (mostly small), a 2 MB page would waste on average 1 MB per process (3 % in the test, and much more for small programs) and would make `fork` and copy-on-write expensive. Transparent huge pages should be enabled only for the few large-memory jobs (simulations) on the server. The 1 KB page is rejected because its table is four times larger and its TLB coverage four times smaller, for a small saving of 1.5 KB per process.

2.9 Alternative concept: an inverted page table instead of a per-process multi-level table

Space. With a per-process table the total space is O(sum over processes of the size of each process's address-space tables): for 500 processes of 25 MB average, with four-level tables, about 25 MB / 4 KB × 8 B = 50 KB each plus upper levels, say 60 KB × 500 = 30 MB. With an inverted table the space is one entry per physical frame: for 16 GB of RAM and 4 KB frames that is 4,194,304 entries × 16 bytes (page number, process ID, flags, hash link) = 64 MB, *fixed*, regardless of how many processes exist or how large their virtual address spaces are: O(M) for physical memory M against O(sum of process sizes). With a sparse 48-bit address space and few processes the inverted table is larger than the multi-level tables; with many large processes it is smaller.

Translation time. The multi-level table needs a walk of one memory reference per level (4 on x86-64) after a TLB miss: O(levels). The inverted table needs a hash of (process ID, page number), then a search of the collision chain: expected O(1) references (about 1 to 2 with a good hash), but one more memory reference for the hash anchor table; worst case

O(chain length). So a lookup is similar or slightly faster than a 4-level walk, but sharing a frame among processes needs extra structures (an entry has only one virtual-page field), the table cannot hold pages that are not in memory (so a separate external page table is still needed for paging out), and the hardware must support it (used by PowerPC and IA-64 in the past). Modern x86 and ARM keep the multi-level design for flexibility and sharing, with large TLBs to hide the walk.

Q3 – Segmentation (6 Marks)

3.1 The problem

Students' programs have code that should be read-only and shared by everyone running the same compiler, a stack that grows, and data of varying size. Paging treats all of these the same way, as arbitrary pages.

3.2 Segmentation and address translation

Figure 3.1 Address translation with a segment table

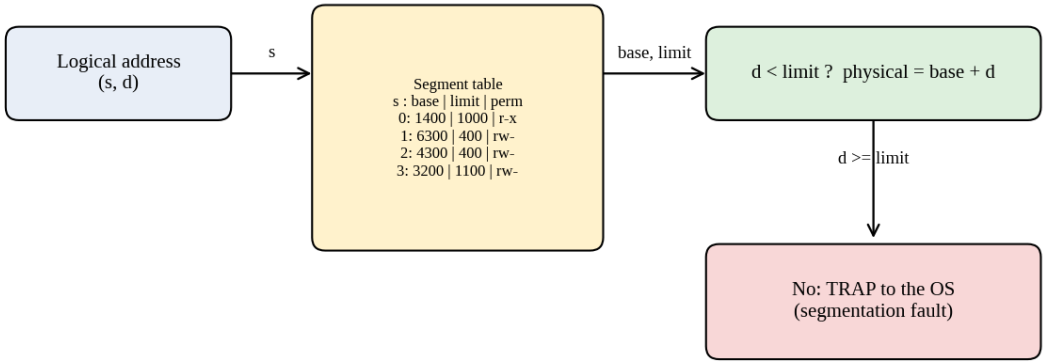


Figure 3.1 Translation with a segment table

In segmentation the logical address space is a collection of *segments*: code, data, heap, stack, libraries. Each segment has a name (number) and a length. A logical address is a pair (segment number s, offset d). The **segment table** contains for each segment the *base* (starting physical address) and the *limit* (length), and often the protection bits. The hardware checks d < limit; if so the physical address is base + d; otherwise it traps to the OS (segmentation fault).

Program: translate eight addresses and show a limit violation

The segment table is: code (base 1,400, limit 1,000, r-x), data (6,300, 400, rw-), heap (4,300, 400, rw-), stack (3,200, 1,100, rw-), lib (4,700, 1,000, r-x). The program translates 8

(segment, offset, operation) triples.

Listing: q3_seg.c

```
/* Segmentation: translate (segment, offset) with a segment table of base and limit; \
       detect limit violations. */
#include <stdio.h>
struct seg { const char *name; unsigned base, limit; const char *perm; };
struct seg table[] = {
    {"code ", 1400, 1000, "r-x"},
    {"data ", 6300,  400, "rw-"},
    {"heap ", 4300,  400, "rw-"},
    {"stack", 3200, 1100, "rw-"},
    {"lib  ", 4700, 1000, "r-x"},
};
int main(void)
{
    struct { int s; unsigned off; char op; } t[] = {
        {0, 53, 'x'}, {1, 852, 'r'}, {2, 399, 'w'}, {3, 1099, 'w'}, {4, 0, 'x'}, {0, \
       1000, 'x'}, {1, 100, 'x'}, {5, 10, 'r'}};
    printf("%-3s %-6s %-5s %-5s  result\n", "#", "seg", "off", "op");
    for (int i = 0; i < 8; i++) {
        printf("%-3d ", i + 1);
        if (t[i].s >= 5) { printf("%-6d %-5u %-5c  TRAP: invalid segment number\n",  \
      t[i].s, t[i].off, t[i].op); continue; }
        struct seg *g = &table[t[i].s];
        printf("%-6s %-5u %-5c  ", g->name, t[i].off, t[i].op);
        if (t[i].off >= g->limit) { printf("TRAP: offset %u >= limit %u (segmentatio \
      n fault)\n", t[i].off, g->limit); continue; }
        char need = t[i].op;
        int ok = need == 'r' ? g->perm[0] == 'r' : need == 'w' ? g->perm[1] == 'w' : \
       g->perm[2] == 'x';
        if (!ok) { printf("TRAP: protection violation (segment is %s)\n", g->perm);  \
      continue; }
        printf("physical %u + %u = %u\n", g->base, t[i].off, g->base + t[i].off);
    }
    return 0;
}
```

$ cd q3 && ./q3_seg

```
#   seg    off   op     result
1   code   53    x      physical 1400 + 53 = 1453
2   data   852   r      TRAP: offset 852 >= limit 400 (segmentation fault)
3   heap   399   w      physical 4300 + 399 = 4699
4   stack  1099  w      physical 3200 + 1099 = 4299
5   lib    0     x      physical 4700 + 0 = 4700
6   code   1000  x      TRAP: offset 1000 >= limit 1000 (segmentation fault)
7   data   100   x      TRAP: protection violation (segment is rw-)
8   5      10    r      TRAP: invalid segment number
```

Verification by hand

(1) code, offset 53: 53 < 1,000, physical 1,400 + 53 = 1,453. (2) data, offset 852: 852 >= 400, trap. (3) heap, offset 399: 399 < 400, physical 4,300 + 399 = 4,699 (the last valid byte). (4) stack, offset 1,099: 1,099 < 1,100, physical 3,200 + 1,099 = 4,299. (5) lib, offset 0, execute: allowed, physical 4,700. (6) code, offset 1,000: equal to the limit, so out of range (valid offsets are 0 to 999), trap. (7) data, execute: offsets within the limit but the segment is not executable, protection trap (this protects against code injection into the data segment). (8) segment 5 does not exist in a table with 5 entries: invalid segment number. All eight results agree with the program.

What happens when an offset exceeds the segment limit

The hardware compares the offset with the limit *before* forming the physical address. If it is exceeded, the CPU raises an exception (a trap) and control goes to the kernel's handler. The OS cannot repair the access, so it sends the signal SIGSEGV to the process, whose default action terminates the process (with a core dump), and the faulty program cannot disturb other segments, as the next section shows with a real process.

3.3 Segmentation with paging (Intel x86)

Figure 3.2 Segmentation with paging (Intel x86 style)

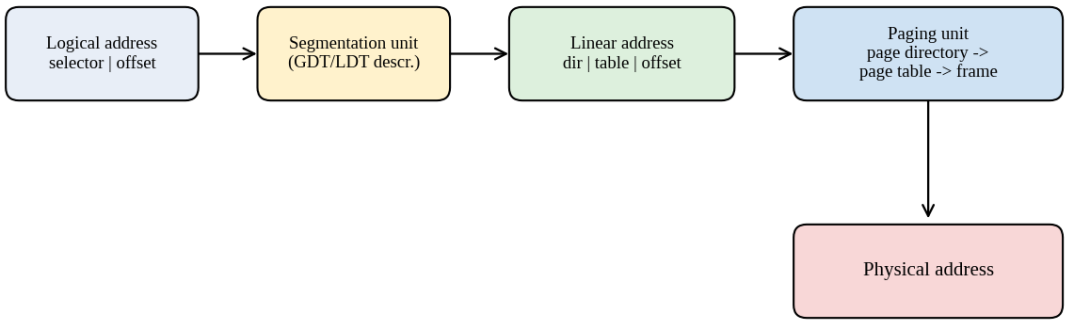


Figure 3.2 Segmentation with paging (Intel style)

The Intel architecture combines the two. The CPU generates a logical address (selector, offset). The **segmentation unit** uses the selector to find a descriptor in the global or local descriptor table, checks the limit and the privilege, and adds the base to form a *linear address*. The **paging unit** then translates the linear address (a directory index, a table index and an offset) through the page tables into a physical address. Segmentation thus provides the programmer's view and protection by segment, and paging provides the fixed-size allocation of physical memory without external fragmentation. 64-bit x86 (long mode) practically disables segmentation (the base of the code, data and stack segments is forced to 0 and limits are ignored, except the FS and GS registers used for thread-local storage), and

Linux relies on paging alone.

3.4 Segments of a running Linux program

On Linux a process's address space is a set of *mappings* shown in `/proc/<pid>/maps`: the executable's code (r-xp) and read-only data (r--p) and data (rw-p), the heap, the shared libraries (each again in code and data parts), anonymous mappings, and the stack:

```
$ cd q3 && bash maps.sh
```

```
segments (mappings) of a running process, pid 1225:
55ad27dcf000-55ad27dd1000 r--p /usr/bin/sleep
55ad27dd1000-55ad27dd5000 r-xp /usr/bin/sleep
55ad27dd5000-55ad27dd6000 r--p /usr/bin/sleep
55ad27dd6000-55ad27dd7000 r--p /usr/bin/sleep
55ad27dd7000-55ad27dd8000 rw-p /usr/bin/sleep
55ad38f33000-55ad38f54000 rw-p [heap]
7f89ff9a7000-7f89ffa00000 r--p /usr/lib/locale/C.utf8/LC_CTYPE
7f89ffa00000-7f89ffa28000 r--p /usr/lib/x86_64-linux-gnu/libc.so.6
7f89ffa28000-7f89ffbb0000 r-xp /usr/lib/x86_64-linux-gnu/libc.so.6
7f89ffbb0000-7f89ffbff000 r--p /usr/lib/x86_64-linux-gnu/libc.so.6
7f89ffbff000-7f89ffc03000 r--p /usr/lib/x86_64-linux-gnu/libc.so.6
7f89ffc03000-7f89ffc05000 rw-p /usr/lib/x86_64-linux-gnu/libc.so.6
7f89ffc05000-7f89ffc12000 rw-p
7f89ffc5e000-7f89ffc61000 rw-p
```

Each line gives the start and end address, the permissions and the file. These are the segments of the modern process, implemented with pages: the permission differences between adjacent regions demonstrate that code is read-only and executable, data is writable, and the stack and heap are writable but not executable.

3.5 Triggering a segmentation fault

Listing: q3_segv.c

```
/* Trigger a segmentation fault and show what the OS reports to the process (SIGSEGV \
       handler). */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <signal.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>
static void handler(int sig, siginfo_t *si, void *ctx)
{
    (void)ctx;
    char msg[128];
    int n = snprintf(msg, sizeof msg, "OS delivered signal %d (SIGSEGV) for address  \
      %p, si_code=%s\n", sig, si->si_addr,
                     si->si_code == SEGV_MAPERR ? "SEGV_MAPERR (address not mapped)" \
       : "SEGV_ACCERR (permission violation)");
```

```
    if (write(1, msg, n) < 0) {}
    _exit(139);
}
int main(int argc, char **argv)
{
    struct sigaction sa; memset(&sa, 0, sizeof sa); sa.sa_sigaction = handler; sa.sa \
      _flags = SA_SIGINFO; sigaction(SIGSEGV, &sa, NULL);
    if (argc > 1) { char *p = (char *)"constant string"; printf("writing into read-o \
      nly code/data segment...\n"); fflush(stdout); p[0] = 'X'; }
    else { volatile int *q = (int *)0x10; printf("reading address 0x10 (outside any  \
      segment)...\n"); fflush(stdout); return *q; }
    return 0;
}
```

$ cd q3 && ./q3_segv

```
reading address 0x10 (outside any segment)...
OS delivered signal 11 (SIGSEGV) for address 0x10, si_code=SEGV_MAPERR (address not  \
      mapped)
```

$ cd q3 && ./q3_segv ro

```
writing into read-only code/data segment...
OS delivered signal 11 (SIGSEGV) for address 0x559973620095, si_code=SEGV_ACCERR (pe \
      rmission violation)
```

In the first run the program reads address 0x10, which is in no mapping (the first page of the address space is deliberately left unmapped to catch null-pointer dereferences). The MMU finds no valid page-table entry and raises a page-fault exception; the kernel's page-fault handler looks up the address in the process's list of memory regions, finds none, decides the reference is illegal and sends SIGSEGV with the code `SEGV_MAPERR`. The program had installed a handler that printed the faulting address and exited with status 139 (128 + 11, the shell's convention for death by signal 11). In the second run the program tried to write into a string literal that lives in a read-only segment: the address is mapped but the permission is violated, so the code is `SEGV_ACCERR`. Without a handler the default action terminates the process and the shell prints "Segmentation fault".

3.6 Fragmentation after loading and removing 20 segments

The simulation loads 20 segments of random sizes (20 to 120 KB) into 1,024 KB with first fit, removes 10 of them at random and measures the free space.

$ cd q3 && python3 q3_frag.py

```
20 segments loaded, sizes (KB): [61, 39, 70, 103, 26, 29, 88, 32, 66, 94, 27, 84, 47 \
      , 24, 31, 75, 73, 28, 50, 31]  total = 1078 KB
after removing 10 segments: free holes = [(0, 100), (170, 103), (416, 32), (514, 121 \
      ), (719, 102), (969, 55)]
  total free = 513 KB, largest hole = 121 KB, number of holes = 6
  external fragmentation = 1 - largest/total = 76.4 %
```

```
  a new segment of 122 KB (smaller than the 513 KB free) can be placed: False
  after compaction one hole of 513 KB would exist (cost: copy 511 KB of live segment \
      s)

Table sizes for a 4 GB process using 4 segments: segment table = 4 entries x 8 bytes \
       = 32 bytes
Single-level page table (4 KB pages, 4 GB): 1,048,576 entries x 4 bytes = 4 MB; two- \
      level: 4 KB top + used 4 KB leaf tables
```

After the removals the 513 KB of free memory is divided into six holes (100, 103, 32, 121, 102 and 55 KB); the largest hole is only 121 KB, so the external fragmentation is 1 - 121/513 = **76.4 %** and a 122 KB segment cannot be placed although 513 KB is free. Compaction would solve it but costs the copying of 511 KB of live segments.

3.7 Comparison of the table sizes and of the three schemes

Scheme	Table size	External fragmentation	Internal fragmentation	Sharing and protection	Translation cost
Pure paging	Large: 4 MB (single level, 32-bit, 4 KB pages) or a multi-level tree	None	Yes, about half a page per process	Per page: bits for each page; sharing a library means sharing many pages	One table lookup (several levels) + TLB
Pure segmentation	Tiny: one entry per segment (for example 4 segments x 8 bytes = 32 bytes)	Yes, severe (76 % in the test)	None	Natural: a whole segment (code) is shared and protected as one unit	One add and one compare; but allocation of variable-size segments is complex
Segmented paging (x86)	Segment table + page tables for each segment	None (paged)	Yes (small)	Segment-level protection plus page-level protection	Two translations (segment then page); TLB hides it

3.8 Conclusion: why do modern operating systems rely mainly on paging?

Modern systems rely mainly on paging because (1) it removes external fragmentation, so physical memory is used completely and allocation is a simple matter of finding any free frame; (2) it makes virtual memory simple: fixed-size pages are the natural unit for moving data between memory and disk, whereas swapping whole variable-size segments is slow and needs contiguous space (the experiment showed a 76 % fragmentation of free space); (3) it works well with the TLB, with sharing and copy-on-write per page; (4) the advantages of segmentation (protection by region, sharing code) can be obtained on top of paging by giving *regions* of pages their own permission bits, as the `/proc/maps` listing

shows (code, data, stack and libraries as separate regions). That is why 64-bit processors reduce segmentation to almost nothing, and the operating system presents the programmer's segments as *regions* of a paged address space.

3.9 Alternative concept: pure paging instead of segmentation

Fragmentation. Paging: no external fragmentation, internal fragmentation of about half a page per segment (2 KB per segment with 4 KB pages; with four segments of a program about 8 KB wasted). Segmentation: no internal fragmentation but severe external fragmentation (76.4 % in the experiment).

Size of the tables. A paged 4 GB address space needs a table of 1,048,576 entries (4 MB) when it is one level, or a tree whose size follows the memory used; the segmented program needs only a 32-byte table with four entries. This is the price of paging, reduced by multi-level tables.

Time to translate. Segmentation: one comparison and one addition (O(1), a few gates); paging: a table walk of k levels after a TLB miss (O(k)); with a TLB hit ratio of 99 % the average difference is small (111 ns against 100 ns in the Q2 calculation).

Sharing and protection. In paging the code segment of the compiler is shared by mapping its pages into each process, and read-only protection is set page by page; the OS must keep the set of pages consistent. In segmentation sharing means pointing the segment table entries of several processes to the same base, one entry per segment, which is simpler to describe and protect. Most modern kernels reconstruct this convenience in software (virtual-memory areas, `vm_area_struct` in Linux) on top of paging.

Q4 – Virtual Memory and Demand Paging (8 Marks)

4.1 The problem

A radiologist must run an image-processing program that uses 24 GB of memory on a laptop with 16 GB of RAM. Virtual memory makes this possible, but performance depends on how often the program faults.

4.2 Handling a page fault

Figure 4.1 Steps in handling a page fault

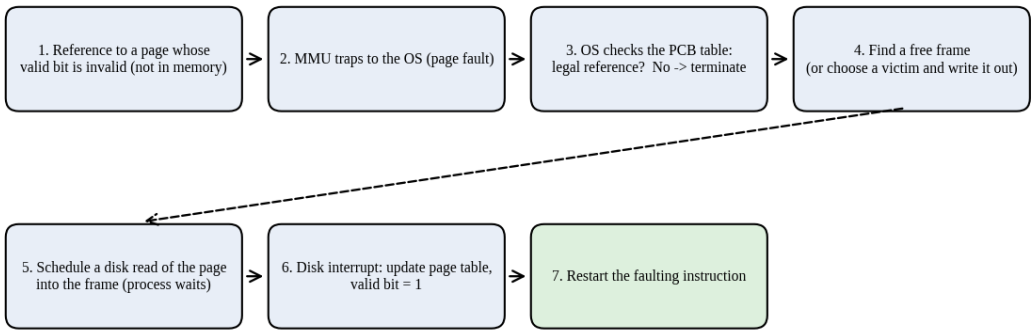


Figure 4.1 Steps in handling a page fault

In **demand paging** a page is brought into memory only when it is referenced. Each page-table entry has a valid–invalid bit. The steps (Figure 4.1) are: (1) the program references a page whose bit is invalid; (2) the MMU traps to the operating system; (3) the OS checks an internal table (in the PCB) to decide whether the reference was illegal (terminate the process) or only the page is not in memory; (4) the OS finds a free frame (if there is none, a page-replacement algorithm chooses a victim, and if the victim is dirty it is written to disk); (5) the OS schedules a disk read of the required page into the frame, and the process waits (another process runs meanwhile); (6) when the I/O completes the OS updates the page table (frame number, valid bit); (7) the faulting instruction is restarted as if nothing had happened. A *minor* fault (the page is already in memory, for example in the page cache, or is a new zero-filled page) costs about 1 to 3 µs; a *major* fault (a disk read) costs about 100 µs on an SSD and 5 to 10 ms on a hard disk.

4.3 Effective access time for different page-fault rates

With memory access time 100 ns and fault service time 8 ms, EAT = (1 - p) × 100 ns + p × 8,000,000 ns.

$ cd q4 && python3 q4_eat.py

```
== Effective access time EAT = (1-p) x 100 ns + p x 8 ms
  p = 1 in 1,000      EAT =    8099.9 ns  (slow-down factor 81.0)
  p = 1 in 100,000    EAT =     180.0 ns  (slow-down factor 1.8)
  p = 1 in 1,000,000  EAT =     108.0 ns  (slow-down factor 1.1)

== Maximum page-fault rate for a 10% slow-down: EAT <= 110 ns -> p <= 10 ns / (8,000 \
      ,000 ns - 100 ns) = 1.250e-06  (about 1 fault in 799990 accesses)

== 24 GB program on 16 GB RAM: if the program touches memory with locality so that p \
```

```
       = 1e-6, EAT = 108.0 ns
== 32 GB machine, no faults after loading: EAT = 100 ns; with p=1e-6 on the small ma \
      chine EAT = 108 ns (8% slower)

== Equal and proportional frame allocation: 62 frames, processes of 10 and 127 pages
  equal: 31 and 31 frames; proportional: 62 x 10/137 = 4.5 -> 4, and 62 x 127/137 =  \
      57.5 -> 57 frames
```

Figure 4.3 Effective access time against page-fault rate

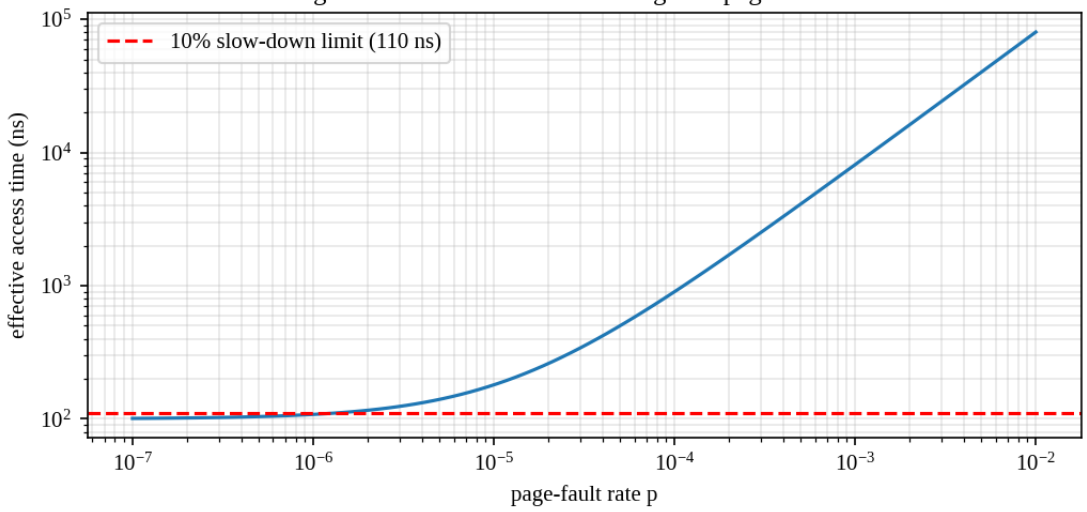


Figure 4.3 Effective access time against page-fault rate

Page-fault rate p	EAT	Slow-down relative to 100 ns
1 in 1,000	8,099.9 ns (about 8.1 µs)	81 times
1 in 100,000	180 ns	1.8 times
1 in 1,000,000	108 ns	1.08 times

The calculation shows how dominated the average is by the rare faults: one fault in a thousand accesses makes the program 81 times slower. **Maximum acceptable page-fault rate for a 10 % slow-down:** EAT <= 110 ns gives (1 - p) × 100 + p × 8,000,000 <= 110, so p <= 10 / 7,999,900 = 1.25 × 10^-6, i.e. fewer than **one fault in 800,000 memory accesses**. A program with a 24 GB working set on 16 GB of RAM can meet this limit only if its access pattern has strong locality.

4.4 Copy-on-write and fast fork

Copy-on-write (COW). When `fork()` creates a child, the OS does not copy the parent's pages. Both processes' page tables point to the same frames, which are marked read-only (and flagged as copy-on-write). When either process writes to a shared page the hardware raises a protection fault; the kernel allocates a new frame, copies the page, maps the copy writable in the writing process, and restarts the instruction. Pages that are never written (code, read-only data, and the parts of the heap that are only read) are never copied. This

makes `fork()` cost proportional to the page-table size and not to the data size, and makes `fork` + `exec` cheap, since the child typically replaces its image straight away.

Experiment. The program `q4_cow.c` allocates and fills 100 MB, forks, and reports (from `/proc/self/smaps_rollup`) the private and shared memory of the child immediately after `fork` and after writing to half of the pages:

Listing: q4_cow.c

```
/* Copy-on-write: fork() a process holding 100 MB, then measure the child's faults a \
      nd private memory before/after writing. */
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <sys/wait.h>
#include <sys/resource.h>
static long minflt(void) { struct rusage r; getrusage(RUSAGE_SELF, &r); return r.ru_ \
      minflt; }
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); ret \
      urn t.tv_sec + t.tv_nsec / 1e9; }
static long field(const char *name) { FILE *f = fopen("/proc/self/smaps_rollup", "r" \
      ); char l[128]; long v = -1; size_t n = strlen(name); while (fgets(l, sizeof l \
      , f)) if (!strncmp(l, name, n)) { v = atol(l + n + 1); break; } fclose(f); ret \
      urn v; }
int main(void)
{
    size_t sz = 100u << 20; char *b = malloc(sz); memset(b, 1, sz);
    printf("parent: Rss %ld KB, Private_Dirty %ld KB\n", field("Rss"), field("Privat \
      e_Dirty")); fflush(stdout);
    double t0 = now(); pid_t p = fork();
    if (p == 0) {
        double t1 = now();
        printf("child  right after fork (fork took %.2f ms): Private_Dirty %ld KB, S \
      hared %ld KB\n", (t1 - t0) * 1e3, field("Private_Dirty"), field("Shared_Dirty" \
      ) + field("Shared_Clean"));
        long f0 = minflt(); double w0 = now();
        for (size_t i = 0; i < sz / 2; i += 4096) b[i] = 2;             /* write to \
       half of the pages */
        double w1 = now();
        printf("child  after writing 50 MB: %ld copy-on-write faults in %.1f ms, Pri \
      vate_Dirty %ld KB\n", minflt() - f0, (w1 - w0) * 1e3, field("Private_Dirty"));
        fflush(stdout); _exit(0);
    }
    waitpid(p, NULL, 0);
    return 0;
}
```

```
$ cd q4 && ./q4_cow
```

```
parent: Rss 103992 KB, Private_Dirty 102512 KB
child  right after fork (fork took 1.18 ms): Private_Dirty 36 KB, Shared 103344 KB
child  after writing 50 MB: 12800 copy-on-write faults in 29.8 ms, Private_Dirty 512 \
      36 KB
```

Right after `fork` (which took 1.18 ms for a 100 MB parent) the child has only **36 KB** of private dirty memory and 103,344 KB shared with the parent: nothing was copied. After the child wrote one byte in each page of the first 50 MB, there were 12,800 copy-on-write faults (50 MB / 4 KB) in 29.8 ms (2.3 µs per fault), and the child's private memory rose to 51,236 KB, exactly the amount written to. The other 50 MB remained shared. This confirms that memory is copied lazily, one page at a time, only when written.

4.5 Thrashing and the working-set model

Thrashing occurs when a process (or the whole system) spends more time paging than executing. The usual cycle: the OS sees low CPU utilisation, so it raises the degree of multiprogramming; the new processes take frames from the others, causing more page faults; processes wait for the paging disk, CPU utilisation falls further, and the OS starts yet more processes. A process thrashes if it has fewer frames than the pages it actively uses.

The **working-set model** is based on locality. The working set WS(t, Δ) of a process is the set of pages referenced in the most recent Δ references (the working-set window). If the total demand D = Σ|WS_i| of all processes is greater than the number of available frames m, thrashing will occur, and the OS must suspend a process (swap it out entirely) until D <= m. If D < m, the OS may start another process. An alternative is **page-fault-frequency (PFF)** control: if a process's fault rate is above an upper threshold it gets more frames; below a lower threshold it loses frames; if no frames are free, a process is suspended. The OS detects thrashing by the combination of low CPU utilisation, a high paging rate and a high disk-queue length.

4.6 How demand paging lets a 24 GB program run in 16 GB, and the allocation of frames

At any moment the program needs only its *working set* (the part of the data it is currently using) in RAM; the rest of the 24 GB stays on disk (swap space or the file that it maps). The program's data are touched slice by slice, and a slice of an image needs only tens of megabytes. When a page is needed that is not in memory a page fault brings it in and a page not recently used (LRU or clock approximation) is evicted. The condition for acceptable speed is that the page-fault rate stays below about 1 in 800,000 accesses (previous section), which means that the working set must be much smaller than the 16 GB RAM and must change slowly.

Number of frames per process. The OS must give each process a minimum number of frames (the number of pages that one instruction can reference; a few). Beyond that: **equal allocation** gives m/n frames to each of n processes; **proportional allocation** gives a_i = (s_i / S) × m frames to a process of size s_i, where S = Σs_i. For 62 free frames and two processes of 10 and 127 pages, equal allocation gives 31 and 31 frames; proportional allocation gives 62 × 10/137 = 4.5, that is 4 frames, and 62 × 127/137 = 57.5, that is 57 frames: the small process receives only the frames that it can use. Priority-based allocation and global versus local replacement (global replacement is more efficient but lets one process steal from another and makes the time of a process depend on the others) are the common refinements; the working-set or PFF methods give each process what it needs.

4.7 Experimental investigation

Page faults for sequential and random touching of memory

The program `q4_touch.c` touches one byte of each of 51,200 pages (200 MB) in sequential order and then in random order, counts the page faults with `getrusage`, and then makes five more passes over the pages.

Listing: q4_touch.c

```
/* Touch 200 MB of memory in sequential and in random order, count page faults and t \
      ime the second pass. */
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <sys/mman.h>
#include <sys/resource.h>
#include <unistd.h>
static long minflt(void) { struct rusage r; getrusage(RUSAGE_SELF, &r); return r.ru_ \
      minflt; }
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); ret \
      urn t.tv_sec + t.tv_nsec / 1e9; }
int main(void)
{
    long ps = sysconf(_SC_PAGESIZE); size_t pages = 51200;       /* 200 MB */
    size_t *perm = malloc(pages * sizeof *perm);
    for (size_t i = 0; i < pages; i++) perm[i] = i;
    srand(3); for (size_t i = pages - 1; i > 0; i--) { size_t j = (size_t)rand() % ( \
      i + 1), t = perm[i]; perm[i] = perm[j]; perm[j] = t; }
    for (int order = 0; order < 2; order++) {
        char *m = mmap(NULL, pages * ps, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_A \
      NONYMOUS, -1, 0);
        long f0 = minflt(); double t0 = now();
        for (size_t i = 0; i < pages; i++) m[(order ? perm[i] : i) * ps] = 1;
        double t1 = now(); long f1 = minflt();
        long sum = 0; for (int rep = 0; rep < 5; rep++) for (size_t i = 0; i < pages \
```

```
      ; i++) sum += m[(order ? perm[i] : i) * ps];
        double t2 = now(); long f2 = minflt();
        printf("%-10s first touch: %6ld faults in %6.1f ms (%.2f us/fault); 5 more p \
      asses: %ld faults, %6.1f ms (checksum %ld)\n", order ? "random" : "sequential" \
      , f1 - f0, (t1 - t0) * 1e3, (t1 - t0) * 1e6 / (f1 - f0), f2 - f1, (t2 - t1) *  \
      1e3, sum);
        munmap(m, pages * ps);
    }
    return 0;
}
```

$ cd q4 && ./q4_touch

```
sequential first touch:  51203 faults in  131.5 ms (2.57 us/fault); 5 more passes: 0 \
       faults,    3.4 ms (checksum 256000)
random     first touch:  51200 faults in   88.3 ms (1.72 us/fault); 5 more passes: 0 \
       faults,    3.4 ms (checksum 256000)
```

The number of page faults is the same for both orders (51,200 plus a few, one per page; the first touch of an anonymous page always faults) and there are **no** faults in the next passes because the pages are now resident. Sequential touching cost 2.57 µs per fault and random 1.72 µs per fault in this run, so the difference between the two orders is *noise and cache effects*, not the number of faults. The order matters when memory is scarce: sequential access reuses a page 1,024 times (4,096 bytes per page / 4 bytes per integer) before moving on, so with demand paging from disk each page is read once; random access across a region larger than memory would fault on a large fraction of the accesses, as calculated below.

Run time as the memory used grows beyond the free RAM

The test machine has no swap, so a program that exceeds RAM is killed by the out-of-memory killer and cannot be timed. To obtain the measurement on a real system the student should run the program in a virtual machine with 1 GB of RAM and a swap partition (or with `systemd-run -p MemoryMax=512M -p MemorySwapMax=2G`), and increase the array from 256 MB to 2 GB. Here the expected behaviour is calculated with the effective-access-time model: for uniformly random access to a data set of D with RAM M the fault rate is p = 1 - M/D. For the radiologist (D = 24 GB, M = 16 GB), p = 33 %, so EAT = 0.33 × 8 ms = 2.7 ms, **26,000 times slower** than memory, a program that would run for one minute would need 18 days. With strong locality (90 % of the accesses to a hot 10 % of the data that fits in memory, the other 10 % random over the remaining 21.6 GB with 13.6 GB of free frames) p = 0.1 × (1 - 13.6/21.6) = 3.7 %, EAT = 296 µs, still 2,960 times slower. Only with sequential, blocked access and read-ahead (where one disk request brings in 128 KB for 32 consecutive pages) does the effective fault rate fall to p ~ 1/32 of the page touches *per page* and, since each page is used for thousands of accesses, to well below 10^-6 per access.

Plot: page faults against the number of frames for a reference string

Figure 4.2 Page faults against the number of frames for a reference string

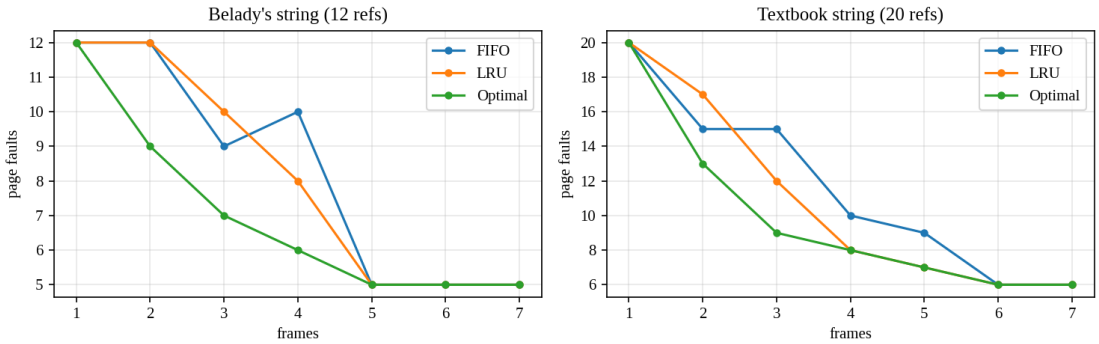


Figure 4.2 Page faults against the number of frames for two reference strings

The graph shows the number of page faults of FIFO, LRU and Optimal for 1 to 7 frames for two reference strings. In general more frames mean fewer faults, but the FIFO line for Belady's string rises from 9 faults with 3 frames to 10 with 4 frames (the anomaly of Q5), whereas the LRU and Optimal curves never rise.

4.8 Conclusion: more RAM or better locality?

For the radiologist's program, **improving locality is the better first solution, and more RAM the better second solution if the working set cannot be reduced**. The calculation shows that the *page-fault rate* decides the speed: a 33 % or 3.7 % fault rate (random or half-local access) makes the program thousands of times slower, whereas a rate below 1 in 800,000 gives less than 10 % slow-down. A change in the algorithm to process the 3D image *in blocks (tiles) or slice by slice*, so that every block is loaded once, processed completely and then replaced, converts the access pattern into the sequential one with a working set of a few hundred MB, and costs nothing but programming time. It also scales to a 64 GB image, which no amount of RAM bought for this laptop would. If the software cannot be changed, **16 GB of extra RAM** (a 32 GB laptop) is a cheap solution, costing the order of 50 to 100 US dollars, against hours of the radiologist's waiting time, and is the right answer when the budget for engineering is zero. Best practice combines the two: a blocked algorithm and enough RAM to hold the working set with margin.

4.9 Alternative concept: a machine with 32 GB RAM instead of relying on demand paging

Effective access time. On the 32 GB machine all 24 GB of data fit in RAM; after the initial load there are no page faults (p = 0), so EAT = 100 ns. On the 16 GB laptop with the measured behaviour of a poorly-localised program, p can be 1 in 1,000 or worse, giving 8,100 ns (81 times slower); with a well-localised program p = 10^-6 and EAT = 108 ns (8 % slower). So with a well-localised program the 32 GB machine gains only 8 %; with a

badly localised program it gains a factor of 81 or more.

Cost of the extra memory against the cost of the slower run. The extra 16 GB of RAM costs a one-time amount of the order of 50 to 100 US dollars. If the poorly localised program runs 81 times slower, a 10-minute analysis takes 13.5 hours, so the cost of the slower run (the radiologist's time, delayed diagnoses) is thousands of dollars within a week. Even the 8 % slow-down of a well-localised program means 48 seconds on a 10-minute job. For a laptop that cannot take more RAM (soldered memory), the remedy has to be software (locality), and the cloud (a 64 GB virtual machine for a few cents per hour) is an alternative. Space complexity: the whole data set needs O(D) of RAM with 32 GB, and only O(W) (the working set) with demand paging.

Q5 – Page-Replacement Techniques (8 Marks)

5.1 The problem

A web server keeps frequently used pages of data in memory. When memory is full the OS must choose which page to remove, and a poor choice causes slow responses.

5.2 The algorithms and the completed program

FIFO replaces the page that has been in memory longest (a queue of pages): simple, but it can remove a heavily used page just because it is old. **Optimal (OPT, Belady's MIN)** replaces the page that will not be used for the longest time in the future: it gives the minimum possible number of faults, but needs knowledge of the future, so it cannot be implemented in a real OS and serves as a benchmark. **LRU** replaces the page that has not been used for the longest time, using the past as an approximation of the future.

TODO 1 in the function `lru` was completed as follows: if a frame is still free it is used (the page and the current time are stored); otherwise the victim is the frame with the smallest `last[]` value (the oldest last use); the new page replaces it and `last[victim]` is set to the current reference index.

Listing: q5_replace.c

```
#include <stdio.h>
#define MAXF 8
int find(const int f[], int n, int page)
{
    for (int i = 0; i < n; i++) if (f[i] == page) return i;
    return -1;
}
int fifo(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, next = 0, faults = 0;
```

```
    for (int i = 0; i < len; i++) {
        if (find(f, used, ref[i]) >= 0) continue;
        faults++;
        if (used < frames) f[used++] = ref[i];
        else { f[next] = ref[i]; next = (next + 1) % frames; }
    }
    return faults;
}
int lru(const int ref[], int len, int frames)
{
    int f[MAXF], last[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        int k = find(f, used, ref[i]);
        if (k >= 0) { last[k] = i; continue; }
        faults++;
        /* TODO 1 (completed) */
        if (used < frames) { f[used] = ref[i]; last[used] = i; used++; }   /* a fram \
      e is free */
        else {
            int victim = 0;                                                /* least  \
      recently used = smallest last[] */
            for (int j = 1; j < frames; j++) if (last[j] < last[victim]) victim = j;
            f[victim] = ref[i]; last[victim] = i;
        }
    }
    return faults;
}
int optimal(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        if (find(f, used, ref[i]) >= 0) continue;
        faults++;
        if (used < frames) { f[used++] = ref[i]; continue; }
        int victim = 0, farthest = -1;
        for (int j = 0; j < frames; j++) {
            int next = len; /* never used again */
            for (int t = i + 1; t < len; t++) if (ref[t] == f[j]) { next = t; break; \
       }
            if (next > farthest) { farthest = next; victim = j; }
        }
        f[victim] = ref[i];
    }
    return faults;
}
int main(void)
{
    int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
    int len = 20;
    for (int fr = 3; fr <= 4; fr++)
        printf("%d frames: FIFO %d, LRU %d, Optimal %d\n", fr, fifo(ref, len, fr),
```

```
            lru(ref, len, fr), optimal(ref, len, fr));
    int b[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
    printf("Belady string, FIFO: 3 frames %d faults, 4 frames %d faults\n",
           fifo(b, 12, 3), fifo(b, 12, 4));
    return 0;
}
```

$ cd q5 && gcc -std=c11 -Wall -Wextra -o q5_replace q5_replace.c && ./q5_replace

```
3 frames: FIFO 15, LRU 12, Optimal 9
4 frames: FIFO 10, LRU 8, Optimal 8
Belady string, FIFO: 3 frames 9 faults, 4 frames 10 faults
```

The output agrees with the expected output: for the textbook string FIFO, LRU and Optimal produce 15, 12 and 9 faults with 3 frames and 10, 8 and 8 with 4 frames; for Belady's string FIFO produces 9 faults with 3 frames and **10** with 4 frames.

Hand verification of the textbook string with 3 frames

$ cd q5 && python3 q5_trace.py

```
FIFO, 3 frames: 15 faults
  ref :  7  0  1  2  0  3  0  4  2  3  0  3  2  1  2  0  1  7  0  1
  flt :  X  X  X  X  .  X  X  X  X  X  X  .  .  X  X  .  .  X  X  X

LRU, 3 frames: 12 faults
  ref :  7  0  1  2  0  3  0  4  2  3  0  3  2  1  2  0  1  7  0  1
  flt :  X  X  X  X  .  X  .  X  X  X  X  .  .  X  .  X  .  X  .  .

OPT, 3 frames: 9 faults
  ref :  7  0  1  2  0  3  0  4  2  3  0  3  2  1  2  0  1  7  0  1
  flt :  X  X  X  X  .  X  .  X  .  .  X  .  .  X  .  .  .  X  .  .
```

Check of FIFO by hand (frames listed oldest first): 7 fault [7]; 0 fault [7,0]; 1 fault [7,0,1]; 2 fault, replaces 7 [0,1,2]; 0 hit; 3 fault, replaces 0 [1,2,3]; 0 fault, replaces 1 [2,3,0]; 4 fault, replaces 2 [3,0,4]; 2 fault, replaces 3 [0,4,2]; 3 fault, replaces 0 [4,2,3]; 0 fault, replaces 4 [2,3,0]; 3 hit; 2 hit; 1 fault, replaces 2 [3,0,1]; 2 fault, replaces 3 [0,1,2]; 0 hit; 1 hit; 7 fault, replaces 0 [1,2,7]; 0 fault, replaces 1 [2,7,0]; 1 fault, replaces 2 [7,0,1]. The faults are at positions 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 14, 15, 18, 19 and 20: **15 faults**, equal to the C program and to the X marks of the trace above (LRU 12 and Optimal 9 agree in the same way).

5.3 Belady's anomaly

Explanation. With the string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, FIFO has 9 faults with 3 frames and 10 with 4 frames. With 3 frames: the pages in memory when 1, 2 come again (positions 5 and 6) are 2, 3, 4 and then the sequence 1, 2 faults; but with 4 frames the pages 1, 2, 3, 4 stay in memory until 5 arrives, which evicts the oldest page, 1; the following 1 then evicts

2, 2 evicts 3, 3 evicts 4, 4 evicts 5 and 5 evicts 1, so the *ageing order of FIFO has nothing to do with the usefulness of the pages*: with 4 frames the pages that FIFO throws away happen to be exactly the ones needed next. FIFO does not consider use, so giving it more memory can change its eviction order for the worse.

Why LRU cannot behave like this: the stack property. An algorithm is a *stack algorithm* if for every reference string the set of pages in memory with n frames is always a subset of the set with n + 1 frames. LRU has this property: the pages in memory with n frames are the n most recently used distinct pages, and these are always among the n + 1 most recently used. Therefore every hit with n frames is also a hit with n + 1 frames, and the number of faults can never increase when frames are added. Optimal has the same property; FIFO does not, since the set of the n oldest-loaded pages is not necessarily a subset of the n + 1 oldest-loaded ones after different eviction decisions.

5.4 Why Optimal is the best but cannot be implemented

OPT needs the complete future reference string at every fault, which the OS cannot know (it depends on the input data and on the program's branches). It is nevertheless useful as a yardstick: in the experiment below, an algorithm that comes close to OPT is nearly as good as can be. Its simulation costs O(frames × n) per fault in the naive form.

5.5 Hardware support for LRU and the clock algorithm

Exact LRU needs an ordering of all pages on every memory reference: either a **counter (time stamp)** field in each page-table entry updated on every access (and a search for the minimum at replacement), or a **stack (doubly linked list)** with the referenced page moved to the top on every reference (six pointer changes per reference). Both need hardware to do work on *every* memory access, which no common processor provides; a software implementation would need a trap on every reference. Real systems therefore use the single **reference bit** that the hardware sets to 1 on any access to the page.

Clock (second-chance) algorithm. The frames are arranged in a circle with a hand. When a victim is needed the hand looks at the page it points to: if its reference bit is 0 the page is replaced; if it is 1 the bit is cleared to 0 (the page gets a second chance) and the hand moves on. A page that is used regularly will have its bit set again before the hand returns; a page that is not used loses it. In the worst case (all bits set) the hand makes a full turn and the algorithm degenerates into FIFO. The extra cost is one bit per frame and the hand, and no work on a hit except the hardware setting the bit.

5.6 Experimental investigation

The program `q5_exp.c` generates two reference strings of 100,000 references over 50 pages: **strong locality** (a working set of 6 pages that moves to a new place every 500 references, with 4 % random noise) and **random** access (uniform over 50 pages), and counts the faults of FIFO, LRU, Clock and Optimal for 3 to 10 frames.

Listing: q5_exp.c

```
/* Page-replacement experiment: 100,000 references, 50 pages; strong locality vs ran \
      dom; FIFO, LRU, Clock, Optimal; 3..10 frames. */
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#define N 100000
#define PAGES 50
static int ref[N], nextuse[N];
static volatile long sink;
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); ret \
      urn t.tv_sec + t.tv_nsec / 1e9; }
static int find(const int *f, int n, int p) { for (int i = 0; i < n; i++) if (f[i] = \
      = p) return i; return -1; }
static int fifo(int fr) { int f[16], used = 0, nx = 0, flt = 0; for (int i = 0; i <  \
      N; i++) { if (find(f, used, ref[i]) >= 0) continue; flt++; if (used < fr) f[us \
      ed++] = ref[i]; else { f[nx] = ref[i]; nx = (nx + 1) % fr; } } return flt; }
static int lru(int fr) { int f[16], last[16], used = 0, flt = 0; for (int i = 0; i < \
       N; i++) { int k = find(f, used, ref[i]); if (k >= 0) { last[k] = i; continue; \
       } flt++; if (used < fr) { f[used] = ref[i]; last[used++] = i; } else { int v  \
      = 0; for (int j = 1; j < fr; j++) if (last[j] < last[v]) v = j; f[v] = ref[i]; \
       last[v] = i; } } return flt; }
static int clock_(int fr) { int f[16], bit[16], used = 0, hand = 0, flt = 0; for (in \
      t i = 0; i < N; i++) { int k = find(f, used, ref[i]); if (k >= 0) { bit[k] = 1 \
      ; continue; } flt++; if (used < fr) { f[used] = ref[i]; bit[used++] = 1; } els \
      e { while (bit[hand]) { bit[hand] = 0; hand = (hand + 1) % fr; } f[hand] = ref \
      [i]; bit[hand] = 1; hand = (hand + 1) % fr; } } return flt; }
static int opt(int fr) { int f[16], nu[16], used = 0, flt = 0; for (int i = 0; i < N \
      ; i++) { int k = find(f, used, ref[i]); if (k >= 0) { nu[k] = nextuse[i]; cont \
      inue; } flt++; if (used < fr) { f[used] = ref[i]; nu[used++] = nextuse[i]; } e \
      lse { int v = 0; for (int j = 1; j < fr; j++) if (nu[j] > nu[v]) v = j; f[v] = \
       ref[i]; nu[v] = nextuse[i]; } } return flt; }
static void prepare(void) { int last[PAGES]; for (int p = 0; p < PAGES; p++) last[p] \
       = N; for (int i = N - 1; i >= 0; i--) { nextuse[i] = last[ref[i]]; last[ref[i \
      ]] = i; } }
static void gen_locality(void) { srand(11); int base = 0; for (int i = 0; i < N; i++ \
      ) { if (i % 500 == 0) base = rand() % PAGES; ref[i] = (base + rand() % 6) % PA \
      GES; if (rand() % 100 < 4) ref[i] = rand() % PAGES; } }   /* working set of 6  \
      pages that moves; 4% noise */
static void gen_random(void) { srand(12); for (int i = 0; i < N; i++) ref[i] = rand( \
      ) % PAGES; }
int main(int argc, char **argv)
{
```

```
    int csv = argc > 1;
    for (int w = 0; w < 2; w++) {
        if (w == 0) gen_locality(); else gen_random();
        prepare();
        if (!csv) printf("\n%s reference string (%d references over %d pages): page  \
      faults\n%-7s %8s %8s %8s %8s\n", w ? "RANDOM" : "STRONG-LOCALITY", N, PAGES, " \
      frames", "FIFO", "LRU", "Clock", "Optimal");
        for (int fr = 3; fr <= 10; fr++) {
            if (csv) printf("%s,%d,%d,%d,%d,%d\n", w ? "random" : "locality", fr, fi \
      fo(fr), lru(fr), clock_(fr), opt(fr));
            else printf("%-7d %8d %8d %8d %8d\n", fr, fifo(fr), lru(fr), clock_(fr), \
       opt(fr));
        }
        if (!csv) {
            double t; printf("running time for %d references, 8 frames (milliseconds \
      ):", N);
            t = now(); for (int r = 0; r < 20; r++) sink += fifo(8); printf(" FIFO % \
      .2f", (now() - t) * 1e3 / 20);
            t = now(); for (int r = 0; r < 20; r++) sink += lru(8); printf(", LRU %. \
      2f", (now() - t) * 1e3 / 20);
            t = now(); for (int r = 0; r < 20; r++) sink += clock_(8); printf(", Clo \
      ck %.2f", (now() - t) * 1e3 / 20);
            t = now(); for (int r = 0; r < 20; r++) sink += opt(8); printf(", Optima \
      l %.2f\n", (now() - t) * 1e3 / 20);
        }
    }
    return 0;
}
```

$ cd q5 && ./q5_exp

```
STRONG-LOCALITY reference string (100000 references over 50 pages): page faults
frames      FIFO      LRU    Clock  Optimal
3          54283    54071    54137    34577
4          40038    39021    39901    21862
5          27040    24468    26060    12754
6          16551    11303    14161     6577
7          11929     6111     8899     4349
8           9473     4661     6312     3828
9           8046     4300     5242     3482
10          7105     4147     4682     3204
running time for 100000 references, 8 frames (milliseconds): FIFO 1.07, LRU 1.13, Cl \
      ock 1.19, Optimal 1.12

RANDOM reference string (100000 references over 50 pages): page faults
frames      FIFO      LRU    Clock  Optimal
3          93978    93983    93979    78974
4          92056    92044    92058    73104
5          90078    90062    90050    68218
6          88062    88077    88048    63958
7          86143    86146    86127    60148
8          84192    84139    84173    56700
```

```
9          82173    82154    82160    53563
10         80174    80194    80170    50658
running time for 100000 references, 8 frames (milliseconds): FIFO 0.64, LRU 1.82, Cl \
      ock 1.56, Optimal 1.85
```

Figure 5.1 Page faults against the number of frames

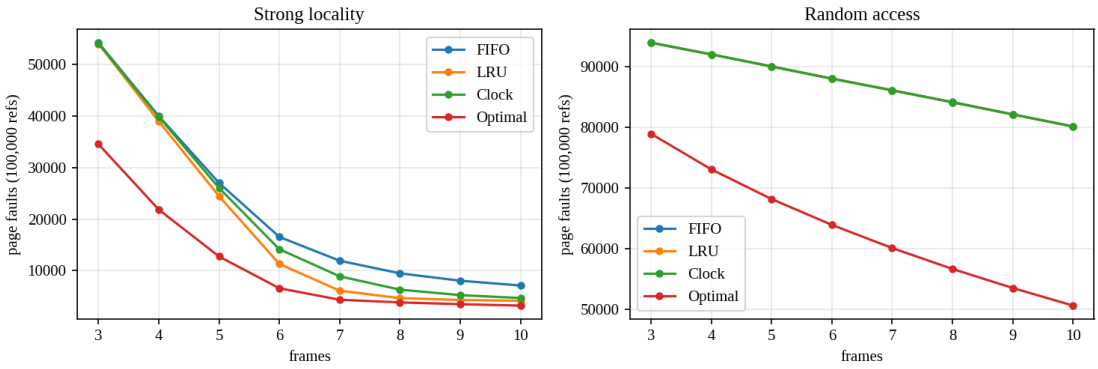


Figure 5.1 Page faults against the number of frames (3 to 10) for each algorithm

Interpretation

- **Strong locality.** With 7 frames LRU has 6,111 faults, Clock 8,899, FIFO 11,929 and Optimal 4,349: FIFO suffers twice the faults of LRU; Clock lies between them (it removes more of the difference at higher frame counts: with 10 frames Clock 4,682, LRU 4,147, FIFO 7,105, Optimal 3,204). The gap between LRU and Optimal (about 40 % with 7 frames) shows that even LRU is not perfect when the working set shifts.
- **Random access.** All three practical algorithms are identical within 0.1 % (93,978 to 93,983 faults with 3 frames), because with no locality no policy can predict the future; they all fault in proportion to 1 - frames/pages (for 10 frames out of 50, about 80 %: 80,174 faults). Only the unrealisable Optimal does better (50,658). So the replacement policy matters *only* when the workload has locality.
- **More frames.** For locality workloads the faults drop sharply once the frames cover the working set (the "knee" at 6 to 8 frames), after which extra frames bring little. A web server should be sized so that the hot set fits.
- **Running time** for 100,000 references with 8 frames (average of 20 runs): FIFO 1.07 ms, LRU 1.13 ms, Clock 1.19 ms and Optimal 1.12 ms for the locality string; 0.64, 1.82, 1.56 and 1.85 ms for the random string. In a simulation with only 8 frames all are cheap (about 10 ns per reference); the interesting cost in a real OS is not CPU time but the hardware support and the cost of the faults: FIFO and Clock need no work on a hit, LRU would need updates on every access.

Hand check with a small string

The 20-reference string with 3 frames was traced by hand (Section 5.2) and by the independent Python script, confirming 15, 12 and 9 faults. The Belady string gives 9 and 10

faults by the program and by the Python function used for Figure 4.2 (FIFO: 3 frames 9, 4 frames 10).

5.7 Engineering decision for the web server

A web server has strong locality (a small set of popular pages accounts for most requests). The recommendation is **LRU or, in practice, the clock (second-chance) algorithm, an LRU approximation**. The measurements show that LRU halves the faults of FIFO for localised workloads (6,111 against 11,929 with 7 frames), that Clock comes much closer to LRU than FIFO does (8,899 and with 10 frames 4,682 against 4,147 for LRU), and that FIFO is the worst and also suffers from Belady's anomaly. Exact LRU is not available without per-access hardware support, so the OS (Linux uses an enhanced clock with active and inactive lists) uses the clock family. For random access the choice does not matter, which supports choosing the cheaper Clock. If the web workload includes large sequential scans (log files, backups) that would flush the cache, a *scan-resistant* variant (two lists, or ARC/LRU-K) should be used.

5.8 Alternative concept: clock instead of exact LRU

Time per reference. Exact LRU must update a time stamp or move a list node on *every* reference: O(1) with a linked list and hash index in software, but it requires a trap or an instruction on every memory access in hardware: about 6 pointer operations per reference for a stack. The clock algorithm sets one bit (done by the hardware at no extra cost); the work is only at a fault: the hand advances over on average k pages (amortised O(1)) and clears bits.

Extra space per frame. Exact LRU: a time stamp of 32 to 64 bits per frame, or two pointers (128 bits) for a list. Clock: **1 bit** per frame (plus one hand pointer for the whole memory), a 32 to 128 times reduction.

Closeness of faults to LRU in the simulation. On the locality string with 3 to 10 frames the clock algorithm is within 0.1 % of LRU for 3 frames (54,137 against 54,071), 6.5 % worse with 5 frames (26,060 against 24,468), and 13 to 46 % worse for 6 to 10 frames (the largest gap is at 7 frames: 8,899 against 6,111; with 10 frames 4,682 against 4,147); for the random string it is within 0.1 %. Clock is clearly closer to LRU than FIFO (11,929 for 7 frames) is. A refinement with two bits (reference and modified) and preference for clean pages gets closer still.

Q6 – Disk Management (4 Marks)

6.1 The problem

The hospital records server must start reliably, survive the failure of a disk and have enough swap space for peak load.

6.2 Formatting, partitioning, boot and bad blocks

Low-level (physical) formatting divides the disk into sectors and writes for each sector a header and trailer with the sector number and an error-correcting code (ECC); it is done by the manufacturer. **Partitioning** divides the disk into one or more groups of cylinders or sectors (partitions) that the OS treats as separate logical disks; the partition table (MBR or GPT) records them. **Logical formatting** creates a file system in a partition: the root directory, the free-space structures and the metadata (for ext4: superblock, inode tables, journal). An unformatted raw partition can be used by databases or for swap.

Boot block and boot process. On power-up the firmware (UEFI or BIOS) runs from ROM, initialises the hardware and reads a small *bootstrap program* from the boot block (the MBR, or the EFI system partition on GPT disks). That loader (GRUB) reads the kernel and an initial RAM disk from the boot partition into memory and starts the kernel; the kernel initialises devices, mounts the root file system and starts `init` (systemd). For a server that must start reliably, the boot loader and the kernel should be on a mirrored boot device, with a fallback kernel entry and the UEFI boot entries for both disks.

Bad blocks. Sectors may fail. Handling: the controller keeps *spare sectors* and remaps a bad sector to a spare transparently (sector sparing) or shifts all following sectors (sector slipping); the OS can mark bad blocks in the file system (`badblocks`, `e2fsck -c`); modern disks and SSDs do this internally and report counts in SMART. Monitoring the growth of reallocated sectors (`smartctl`) gives early warning of a failing disk.

Swap space. Swap is disk space used as the backing store for anonymous memory pages. The size depends on the RAM, the workload and whether hibernation is used: for a records server with 64 GB RAM and no hibernation, 8 to 16 GB (about 12 to 25 % of RAM) is a sensible allowance for peaks, on an SSD (or a mirrored pair) rather than in the database's storage, with a low `vm.swappiness` (10) so the database cache is not swapped needlessly. **Swap on a separate partition or disk versus a swap file:** a partition has no file-system overhead and is contiguous, so it is slightly faster; it is fixed in size and requires planning when the system is installed. A swap *file* is created on the existing file system (`fallocate`, `mkswap`, `swapon`), can be resized without repartitioning and, since Linux 2.6, performs almost as well as a partition (the kernel maps the file's extents), so it is preferred for

flexibility; a **separate disk** gives the advantage that swap I/O does not compete with the database I/O, which is important in a records server under peak load.

6.3 Partitions and swap on the Linux test system

`$ cd q6 && bash sys.sh`

```
$ lsblk
NAME  MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
zram0 253:0    0    0B  0 disk
vda   254:0    0  256G  0 disk /
vdb   254:16   0 12.5M  1 disk /opt/rclone
vdc   254:32   0  696K  1 disk /mnt/skills/public
vdd   254:48   0  5.8M  1 disk /mnt/skills/examples
$ fdisk -l
sys.sh: line 1: fdisk: command not found
$ cat /proc/swaps; swapon --show
Filename                Type        Size        Used        Priority
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.9Gi       319Mi       3.7Gi        15Mi       116Mi       3.6Gi
Swap:             0B          0B          0B
$ df -hT / | head -3
Filesystem     Type  Size  Used Avail Use% Mounted on
/dev/vda       ext4  252G  8.6G   10G  47% /
$ cat /sys/block/vda/queue/scheduler 2>/dev/null || ls /sys/block
none [mq-deadline] kyber bfq
```

`lsblk` shows the block devices: `vda` (256 GB, the root disk with the file system mounted at /), the small read-only images, and a 0-byte `zram0` placeholder. `fdisk -l` is not installed in the sandbox (on a normal system `sudo fdisk -l` prints the partition table: device, start, end, sectors, size, type); `/proc/swaps` and `swapon --show` are empty and `free -h` shows `Swap: 0B`: this machine has **no swap**, which is why the out-of-memory experiments of Q4 and Q7 cannot be run to completion. The file system on the root disk is ext4 and the I/O scheduler of the virtual disk is `mq-deadline`. The student should add the output of `lsblk`, `sudo fdisk -l` and `swapon --show` from their own machine.

6.4 RAID levels, capacity and probability of data loss

`$ cd q6 && python3 q6_raid.py`

```
Usable capacity of 6 disks x 4 TB
  RAID 0                              usable 24 TB (100%)  survives 0              \
                      read 6x  write 6x
  RAID 1 (3 mirrored pairs)           usable 12 TB (50%)  survives 1 (up to 3, one  \
      per pair)      read up to 6x  write 3x
  RAID 5                              usable 20 TB (83%)  survives 1               \
                     read 5x  write about 1.5x (4 I/Os per small write)
  RAID 6                              usable 16 TB (67%)  survives 2               \
                     read 4x  write about 1x (6 I/Os per small write)
```

```
  RAID 10 (3 mirrored pairs striped)   usable 12 TB (50%)  survives 1 per pair (at l \
      east 1, up to 3) read 6x  write 3x

Probability of data loss during one rebuild (a disk has failed; annual failure rate  \
      AFR = 3%, rebuild takes 24 h)
  p(one given surviving disk fails during rebuild) = 8.22e-05
  RAID 1 (pair): the single partner disk must fail:           P = 8.22e-05
  RAID 5 (6 disks): any of the 5 survivors fails:             P = 1-(1-p)^5 = 4.11e- \
      04   (about 5.0x RAID 1)
  RAID 6 (6 disks): two more of the 5 survivors must fail:    P = C(5,2) p^2 = 6.76e \
      -08
  RAID 10 (3 pairs): only the failed disk's partner matters:  P = 8.22e-05
  Rebuild of RAID 5 reads 5 x 4 TB = 2e+14 bits; with URE rate 1e-14 per bit expecte \
      d read errors = 1.60 -> P(at least one) = 80%
  RAID 1/10 rebuild reads only 4 TB = 3.2e+13 bits: P(URE) = 27%

Swap sizing example: 64 GB RAM records server, hibernation not used: swap = 8-16 GB  \
      (safety margin for peaks) on SSD; swappiness 10
```

Figure 6.1 RAID levels compared for 6 disks of 4 TB

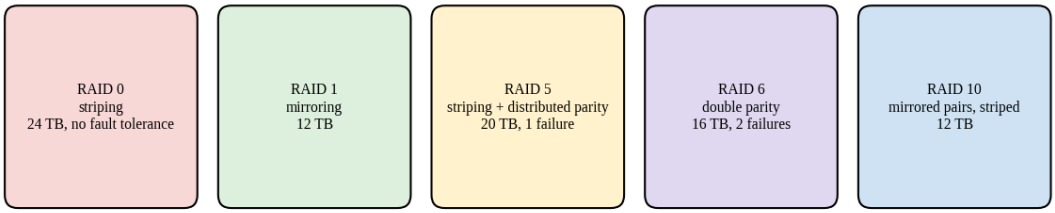


Figure 6.1 RAID levels for 6 disks

Level	Method	Usable capacity (6 x 4 TB)	Failures survived	Read speed	Write speed	Remarks
RAID 0	Striping	24 TB	0	Up to 6x	Up to 6x	No redundancy: any failure loses everything
RAID 1	Mirroring	12 TB (3 pairs)	1 per pair	Up to 6x (reads from either copy)	3x	Simple, fast rebuild (copy of one disk)
RAID 5	Striping with distributed parity	20 TB	1	5x	Small writes need 4 I/Os (read data and parity, write both)	Cheap capacity; long rebuilds, risky with large disks

Level	Method	Usable capacity (6 x 4 TB)	Failures survived	Read speed	Write speed	Remarks
RAID 6	Double distributed parity	16 TB	2	4x	Small writes need 6 I/Os	Survives a second failure during a rebuild
RAID 10	Striped mirrors	12 TB	1 per pair (at least 1)	6x	3x	Best combination of speed and safety; 50 % capacity

Probability of data loss given a disk failure

Assume an annual failure rate (AFR) of 3 % per disk and a rebuild time of 24 hours. The probability that a given surviving disk fails during the rebuild window is p = 0.03 × 24/8760 = 8.2 × 10^-5. **RAID 1** (or the mirror pair in RAID 10) loses data only if the *one* partner disk fails during the rebuild: P = 8.2 × 10^-5. **RAID 5** with 6 disks loses data if *any one of the 5 survivors* fails: P = 1 - (1 - p)^5 = 4.1 × 10^-4, five times higher, and this ignores unrecoverable read errors. **RAID 6** needs two further failures: P = C(5,2) p^2 = 6.8 × 10^-8, about 1,200 times safer than RAID 1/10 for the failure window. Including unrecoverable read errors (URE rate 10^-14 per bit), the rebuild of RAID 5 reads 5 × 4 TB = 1.6 × 10^14 bits, the expected number of errors is 1.6 and the probability of at least one is about 80 % — a read error during the rebuild means data loss (or a damaged file) with large disks; the rebuild of a RAID 1/10 pair reads only 4 TB (27 %).

6.5 Read speed of SSD and hard disk

Only the SSD is available. The `dd` measurement of a 512 MB file gave:

```
$ cd q6 && bash dd.sh
```

```
$ dd if=/dev/zero of=/tmp/tf bs=1M count=512 oflag=direct   (sequential write)
536870912 bytes (537 MB, 512 MiB) copied, 0.538788 s, 996 MB/s
$ dd if=/tmp/tf of=/dev/null bs=1M iflag=direct   (sequential read, no cache)
536870912 bytes (537 MB, 512 MiB) copied, 0.20843 s, 2.6 GB/s
$ dd if=/tmp/tf of=/dev/null bs=1M   (sequential read, through the page cache)
536870912 bytes (537 MB, 512 MiB) copied, 0.263212 s, 2.0 GB/s
```

That is a sequential write of 996 MB/s, a direct read of 2.6 GB/s and, through the page cache, 2.0 GB/s. A hard disk would give roughly 150 to 250 MB/s sequentially and, more importantly, 5 to 10 ms (about 100 I/O per second) for random access against about 100 µs (10,000 or more I/O per second) for an SSD, a factor of 50 to 100 in random-access latency. If a hard disk is available, the student should run `dd if=file of=/dev/null bs=1M iflag=direct` on both and compare.

6.6 Conclusion: recommended storage for the records server

Recommendation: six SSDs (or, for cost, SSDs for the active database and hard disks for the archive) in **RAID 10** for the database and journal, with a mirrored pair (RAID 1) for the boot/OS partition (two EFI boot entries), a swap file of 8 to 16 GB on the OS mirror, hot-spare disks, SMART monitoring, and a nightly backup to a separate system. **Reasons:** the records database has many small random writes (RAID 5 and 6 suffer the read-modify-write penalty of 4 to 6 I/Os per small write; RAID 10 needs 2); it must survive the failure of a disk and rebuild quickly (a RAID 10 rebuild copies one disk, with the lowest exposure: 27 % URE probability against 80 % for RAID 5, and a data-loss probability for the window of 8.2 × 10^-5 against 4.1 × 10^-4); read performance scales with the disks. The price is 50 % capacity (12 TB usable out of 24 TB), which is acceptable for patient records, where safety and speed outweigh the cost of disks. RAID is *not* a backup: deletion or corruption is replicated, so independent backups are needed.

6.7 Alternative concept: RAID 6 instead of RAID 5 or RAID 10

Usable capacity. RAID 6 with 6 disks of 4 TB: (6 - 2) × 4 = **16 TB** (67 %), against 20 TB for RAID 5 and 12 TB for RAID 10.

Time to write a block. In RAID 6 every small write needs the data block and two parity blocks (P and Q, computed with Reed–Solomon arithmetic over a Galois field) to be updated: read old data, old P and old Q, compute the new parities, and write three blocks: **6 I/Os** against 4 for RAID 5 and 2 for RAID 10. The extra parity calculation costs CPU time (O(1) per block for each parity; Q needs a multiplication in GF(2^8) per byte, done with SIMD instructions at several GB/s) but the main cost is the extra disk operations: writes are roughly 1.5 times slower than RAID 5 and 3 times slower than RAID 10. For full-stripe writes the overhead is only the 2/6 of the data written as parity.

Disk failures survived. Two simultaneous failures, against one for RAID 5 and one-per-pair for RAID 10 (two failures in the same mirror pair lose data: probability 1/5 that the second failure hits the partner). During a rebuild RAID 6 still has one parity of protection, so the 80 % read-error risk of the RAID 5 rebuild is no longer fatal.

Judgement. RAID 6 is the better choice for the *archive* of old records (large, mostly read, write-once, capacity matters, and long rebuilds of large disks make double parity attractive). RAID 10 remains the choice for the active database because of its write performance.

Q7 – Design a Memory-Management Plan from First Principles (10 Marks)

7.1 The problem

A smart factory installs edge devices with 2 GB of RAM and no swap disk. Each device runs a real-time control task that must never be delayed by a page fault, a machine-learning task that needs 1.5 GB, and a logging task. The vendor wants to add more tasks in the future. The design is developed from the requirements.

Stage 1 – Interpret the Problem (2 Marks)

Memory needs and timing constraints of each task

Task	**Memory need (assumed from the problem and typical control software)**	**Timing constraint**	**Tolerates page faults?**
Real-time control	Code 5 MB, tables and buffers 15 MB, stack 1 MB; about 20-64 MB in all	Control loop every 1 ms with a deadline of 500 microseconds for each iteration; jitter below 100 microseconds	**No**: a major fault costs 100 us - 10 ms; even a minor fault costs 1-20 us
Machine learning (inference)	1.5 GB (model 300 MB mapped from a file + 1.2 GB heap for tensors)	Batch inference, result within 1-2 s; best effort	Minor faults yes; major faults of file-backed model pages yes if rare; **anonymous heap cannot be swapped (no swap)**
Logging	About 40 MB of buffers; a growing log file on flash	Best effort; must not lose records but may be delayed seconds	Yes
Operating system and kernel	About 300 MB (Linux, services)	none	-

Total: 300 MB (OS) + 64 MB (RT) + 1,500 MB (ML) + 40 MB (logging) = 1,904 MB of the 2,048 MB: **144 MB free** (7 %) with no swap to fall back on.

Requirements as measurable statements

1. R1: The worst-case latency of one iteration of the control loop is **below 500 µs** in 99.999 % of 10 million iterations, with **zero** major page faults and zero minor faults of the real-time task during operation (verified with `getrusage` and the `perf` page-fault

counters).

2. R2: The real-time task's code, data, heap and stack are **locked in RAM** (`mlockall`) and pre-faulted before the control loop starts; the locked amount is at most 64 MB.
3. R3: The machine-learning task can use at most 1.5 GB of RAM, enforced with a memory limit (cgroup `memory.max`), so that it cannot exhaust memory needed by the real-time task.
4. R4: Because there is no swap, the system never runs out of memory: the sum of the limits of all tasks plus the kernel must not exceed 2,048 MB, and the OOM killer must be configured to kill the ML or logging task first (`oom_score_adj` = -1000 for the RT task).
5. R5: Adding a task must not change R1.

Stage 2 – Compare Candidate Mechanisms (3 Marks)

Mechanism	Real-time control task	Machine-learning task	Logging task	Verdict
Contiguous allocation	Deterministic: one block, no translation; but requires a fixed size and no fragmentation; does not exist on a general-purpose Linux (the MMU is always on)	Fragmentation: 1.5 GB contiguous block hard to obtain after the system has run for a while	Wasteful for variable-size growth	Not used (possible on an MCU/RTOS without MMU)
Pure paging	Good, if the pages are pre-faulted and locked: translation cost is a TLB hit (use huge pages to reduce TLB misses)	Good: any frames can be used, no external fragmentation	Good	**Base mechanism** (this is what Linux provides)
Segmentation	Not available as such on x86-64; regions (VMAs) with permissions give the benefits	Regions with different protection (code read-only, model file read-only)	-	Used only as regions on top of paging
Demand paging (virtual memory)	**Harmful** if it can cause faults: page fault = unbounded delay; must be disabled by `mlock`	Useful for the *file-backed* model (300 MB loaded on demand and evictable) but no use for anonymous data without swap	Useful for the log file cache	Use for ML model and log; **disable for RT task**

Locking memory for the real-time task. The task calls `mlockall(MCL_CURRENT | MCL_FUTURE)` at start-up, which pins all current and future pages of the process in RAM so

that they are never evicted; before the control loop begins, it pre-faults its stack and heap (touching every page, so that no minor fault happens in the loop) and allocates all working buffers *in advance* (no `malloc` inside the loop); it also uses `MAP_POPULATE` or `mlock` for specific buffers, runs under the `SCHED_FIFO` real-time scheduling class, is pinned to a dedicated CPU core (`sched_setaffinity`, `isolcpus`), and avoids system calls that may block. The locked memory counts against `RLIMIT_MEMLOCK`, which must be raised for this process (`ulimit -l` or limits.conf; on the test machine it is 8,192 KB by default in a normal account and the process was allowed to lock).

Stage 3 – Implement and Verify (3 Marks)

Program: locking the memory of a time-critical loop

The program `q7_mlock.c` simulates the control task. In each iteration it touches one *new* 4 KB page of a 32 MB data area, as a task that works through new data would, and measures the time of that access. Two data types are tested: anonymous memory (never touched before) and file-backed memory whose pages were evicted from the page cache (so that they need disk I/O). Each is run without locking, and with `mlock` plus pre-fault before the loop.

Listing: q7_mlock.c

```
/* Q7 Stage 3: worst-case delay of a time-critical loop with and without locked memo \
      ry.
   Each iteration reads/updates one new 4 KB page of the controller's data (anonymou \
      s or file-backed, evicted from cache). */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>
#include <sys/mman.h>
#include <sys/resource.h>
#define PAGES 8192                                    /* 32 MB of controller data */
static double now_us(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t);  \
      return t.tv_sec * 1e6 + t.tv_nsec / 1e3; }
static int cmp(const void *a, const void *b) { double x = *(const double *)a, y = *( \
      const double *)b; return x < y ? -1 : x > y; }
static long majflt(void) { struct rusage r; getrusage(RUSAGE_SELF, &r); return r.ru_ \
      majflt; }
static long minflt(void) { struct rusage r; getrusage(RUSAGE_SELF, &r); return r.ru_ \
      minflt; }
static void report(const char *name, double *d, int n, long mn, long mj)
{
    double sum = 0; for (int i = 0; i < n; i++) sum += d[i];
    qsort(d, n, sizeof *d, cmp);
```

```
    printf("%-34s mean %8.2f us  p99 %8.2f us  p99.9 %8.2f us  WORST %9.2f us   mino \
      r faults %5ld  major faults %4ld\n", name, sum / n, d[(int)(n * 0.99)], d[(int \
      )(n * 0.999)], d[n - 1], mn, mj);
}
int main(void)
{
    long ps = 4096; static double d[PAGES];
    /* ---- (1) anonymous memory, first touch of each page */
    for (int lock = 0; lock < 2; lock++) {
        char *m = mmap(NULL, PAGES * ps, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_A \
      NONYMOUS, -1, 0);
        int ok = 1;
        if (lock) { if (mlock(m, PAGES * ps)) { perror("mlock"); ok = 0; } else for  \
      (int i = 0; i < PAGES; i++) m[i * ps] = 0; }   /* lock and pre-fault before th \
      e loop starts */
        long a = minflt(), b = majflt();
        for (int i = 0; i < PAGES; i++) { double t0 = now_us(); m[i * ps] = (char)i; \
       d[i] = now_us() - t0; }
        report(lock ? (ok ? "anonymous data, mlock + pre-fault" : "anonymous data, m \
      lock FAILED") : "anonymous data, no locking", d, PAGES, minflt() - a, majflt() \
       - b);
        munmap(m, PAGES * ps);
    }
    /* ---- (2) file-backed data whose pages have been evicted from the page cache ( \
      major faults) */
    int fd = open("ctl.dat", O_RDWR | O_CREAT | O_TRUNC, 0600); char *z = calloc(1,  \
      1 << 20); for (int i = 0; i < PAGES * ps / (1 << 20); i++) if (write(fd, z, 1  \
      << 20) < 0) return 1;
    fsync(fd);
    for (int lock = 0; lock < 2; lock++) {
        posix_fadvise(fd, 0, 0, POSIX_FADV_DONTNEED);           /* evict the file fr \
      om the page cache */
        char *m = mmap(NULL, PAGES * ps, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
        if (lock) { if (mlock(m, PAGES * ps)) perror("mlock"); }            /* mlock \
       reads every page in now, before the critical loop */
        long a = minflt(), b = majflt();
        for (int i = 0; i < PAGES; i++) { double t0 = now_us(); volatile char c = m[ \
      (size_t)i * ps]; (void)c; d[i] = now_us() - t0; }
        report(lock ? "file-backed (evicted), mlock" : "file-backed (evicted), no lo \
      cking", d, PAGES, minflt() - a, majflt() - b);
        munmap(m, PAGES * ps);
    }
    unlink("ctl.dat");
    return 0;
}
```

```
$ cd q7 && ./q7_mlock
```

```
anonymous data, no locking         mean     1.89 us  p99    26.30 us  p99.9    61.50 \
       us  WORST    172.01 us   minor faults  8211  major faults    0
anonymous data, mlock + pre-fault  mean     0.04 us  p99     0.08 us  p99.9     0.21 \
       us  WORST      3.28 us   minor faults     0  major faults    0
file-backed (evicted), no locking  mean     2.00 us  p99    64.23 us  p99.9   117.68 \
       us  WORST    227.80 us   minor faults   260  major faults    1
file-backed (evicted), mlock       mean     0.19 us  p99     0.42 us  p99.9    29.85 \
       us  WORST     60.32 us   minor faults     0  major faults    0
```

Results. Without locking, the worst iteration took **172 µs** for anonymous memory (8,211 minor faults, p99 = 26 µs) and **228 µs** for the file-backed data (260 minor faults and one major fault; p99 = 64 µs), which is 100 to 400 times larger than the 0.5 to 2 µs of the ordinary iteration and would, in the real machine with a loaded disk, reach milliseconds; with locking and pre-faulting the worst case was **3.28 µs** (anonymous, no faults in the loop, p99.9 = 0.21 µs) and 60 µs for the file-backed case (p99 = 0.42 µs; the remaining 60 µs maximum is the effect of a timer interrupt or scheduling of the virtual machine rather than a page fault, since the fault counters are zero). The improvement of the worst case for anonymous data is a factor of 52 and in the 99th percentile of 330. The measurement was made in a virtual machine with other activity on one core, so the absolute worst-case values include scheduling noise; the on-target test must be repeated with `SCHED_FIFO`, an isolated core and a real load (cyclictest, stress-ng).

Simulating the memory use of the three tasks with a page-replacement simulator

The Python simulator `q7_sim.py` models the 2 GB device: the kernel (300 MB), the locked RT task (64 MB), the ML heap (1,200 MB of anonymous memory, not evictable because there is no swap) and the logging buffers (40 MB) leave **444 MB** (113,664 frames) for evictable (file-backed) pages: the RT task's code, the 300 MB model file and the log file. Two million references are generated (50 % RT, 40 % ML sweeping sequentially through the model file, 10 % logging sequentially) and run through an LRU simulator with only 2 % of those frames (9 MB, to reproduce the memory pressure after a further task is added), once without locking and once with the RT pages pinned.

Listing: q7_sim.py

```
"""Q7 Stage 3: simulate the memory use of the three tasks of the edge device (2 GB R \
      AM, no swap) with a page-replacement simulator.
Pages are 4 KB.  Only FILE-BACKED pages (code, model file, log file) can be evicted: \
       there is no swap for anonymous memory.
Frames available for evictable pages = 2 GB - OS/kernel (300 MB) - locked RT task (6 \
      4 MB) - anonymous heap of ML (1.2 GB) - logging buffers (40 MB)."""
import random, collections
random.seed(5)
MB = 256                      # pages per MB
RAM = 2048 * MB
```

```
kernel, rt_locked, ml_heap, log_buf = 300 * MB, 64 * MB, 1200 * MB, 40 * MB
frames = RAM - kernel - rt_locked - ml_heap - log_buf
print("evictable frames: %d pages = %.0f MB" % (frames, frames / MB))
# reference streams (file-backed): RT code/tables 20 MB hot; ML model file 300 MB re \
      ad in sweeps; logging appends sequentially
def stream(n, pinned):
    rt_pages = list(range(0, 20 * MB, 1)); ml_base = 100000; log_base = 500000
    refs = []; ml_pos = 0; log_pos = 0
    for i in range(n):
        r = random.random()
        if r < 0.50: refs.append(("RT", random.choice(rt_pages[:5 * MB])))           \
             # RT task: hot 5 MB of its 20 MB
        elif r < 0.90: refs.append(("ML", ml_base + ml_pos)); ml_pos = (ml_pos + 1)  \
      % (300 * MB // 4 * 4)  # sequential sweep of the 300 MB model file
        else: refs.append(("LOG", log_base + log_pos)); log_pos += 1                 \
              # sequential log
    return refs
def simulate(refs, nframes, pin_rt):
    lru = collections.OrderedDict(); faults = collections.Counter(); total = collect \
      ions.Counter()
    for task, pg in refs:
        total[task] += 1
        if pin_rt and task == "RT": continue               # locked with mlock(): al \
      ways resident, never faults
        if pg in lru: lru.move_to_end(pg); continue
        faults[task] += 1
        if len(lru) >= nframes: lru.popitem(last=False)
        lru[pg] = 1
    return faults, total
refs = stream(2_000_000, False)
for pin in (False, True):
    # memory pressure: only 8% of the evictable frames are left after the fourth tas \
      k / peak (to expose the effect)
    nf = int(frames * 0.02)
    f, t = simulate(refs, nf, pin)
    print("\nRT task %s, %d frames for evictable pages (%.0f MB):" % ("LOCKED (mlock \
      )" if pin else "NOT locked", nf, nf / MB))
    for k in ("RT", "ML", "LOG"): print("  %-4s references %8d  page faults %8d  fau \
      lt rate %.4f%%" % (k, t[k], f[k], 100 * f[k] / t[k]))
print("\nFourth task of 500 MB added: free memory before = %d MB -> after = %d MB" % \
       (2048 - 300 - 64 - 1200 - 40 - 20, 2048 - 300 - 64 - 1200 - 40 - 20 - 500))
```

$ cd q7 && python3 q7_sim.py

```
evictable frames: 113664 pages = 444 MB

RT task NOT locked, 2273 frames for evictable pages (9 MB):
  RT   references   998896  page faults   331657  fault rate 33.2024%
  ML   references   800746  page faults   800746  fault rate 100.0000%
  LOG  references   200358  page faults   200358  fault rate 100.0000%

RT task LOCKED (mlock), 2273 frames for evictable pages (9 MB):
```

```
  RT   references   998896  page faults        0  fault rate 0.0000%
  ML   references   800746  page faults   800746  fault rate 100.0000%
  LOG  references   200358  page faults   200358  fault rate 100.0000%

Fourth task of 500 MB added: free memory before = 424 MB -> after = -76 MB
```

Verification that the real-time task is never delayed. Unlocked, the RT task suffered 331,657 page faults in 998,896 references (a fault rate of **33 %**) because the streaming ML and logging tasks flush its pages from the small evictable pool; locked, the RT task had **0 faults** in 998,896 references. The ML and logging tasks fault on every reference to the model sweep and the log (100 %) in this stress setting because their sequential sweeps are larger than the memory available to them (LRU's worst case), which is acceptable for them: they are best-effort tasks, and the faults delay only them. This simulation, together with the measured zero faults in the locked loop, verifies requirement R1 (zero faults for the RT task) under heavy pressure.

Stage 4 – Defend the Design (2 Marks)

Justification using the measurements. (1) Demand paging is *harmful* to the control task: unlocked, one iteration took up to 172 to 228 µs in the measurement, a large fraction of the 500 µs budget (in the worst case on a loaded real device a major fault takes milliseconds and breaks the deadline); locking reduced the maximum to 3.3 µs for anonymous data and eliminated all faults (0 minor, 0 major). (2) The page-replacement simulation shows 33 % faults for the unlocked RT task under pressure and 0 % with locking. (3) The memory budget is respected: 1,904 MB of 2,048 MB leave 144 MB free as a safety margin; the ML task is limited by a cgroup so that it cannot take the locked memory; and the ML model is mapped from a file so that its 300 MB can be evicted and re-read, the only part of the ML task that can use demand paging.

What changes if a fourth task of 500 MB is added. The free memory would be 144 - 500 = **-356 MB**: it does not fit, and since there is no swap, the OOM killer would kill a process (the cgroup limits and `oom_score_adj` ensure that it is not the RT task). The options, in order of preference: (a) reduce the ML heap by quantising the model (8-bit weights cut the 1.2 GB heap by 4 times), freeing about 900 MB; (b) memory-map the data of the new task from flash and let it page on demand (file-backed pages can be evicted, so the 500 MB need not all be resident); (c) schedule the tasks in time (not simultaneously); (d) add RAM (a hardware change); (e) compress memory with zram (compressed swap in RAM, typically 2:1 to 3:1, which gives an effective 300 to 600 MB for compressible data; it costs CPU time but never the RT task, which is locked). Whatever is chosen, the RT task's locked 64 MB is not touched, and the admission rule R4 (sum of limits <= 2,048 MB) rejects a new task that does not fit, rather than letting the system fail.

One remaining limitation. `mlock` removes page faults but not other sources of latency: *TLB misses*, *cache interference* from the memory-hungry ML task (shared L3 and memory bandwidth), *interrupt and scheduler jitter*, and memory-bus contention can still add tens of microseconds, as the 60 µs worst case in the file-backed locked run shows. A general-purpose Linux kernel also cannot give a *proved* bound; PREEMPT_RT, isolated cores, cache partitioning (Intel CAT) and huge pages reduce but do not eliminate the problem. A second limitation is the lack of swap: any growth of the ML heap beyond its limit kills the task. These limitations lead directly to the alternative below.

7.2 Alternative concept: a real-time OS with no virtual memory instead of Linux with locked memory

Worst-case delay of the control task. A hard real-time OS (VxWorks, QNX, FreeRTOS on an MMU-less or MPU-protected core) has no demand paging at all: all memory is resident from the start, so a page fault cannot happen, and its scheduler and interrupt handling have bounded worst-case times (a few microseconds), certified in many cases. The worst-case delay of the control task is therefore both smaller (typically 5 to 20 µs) and *provable*, against the measured 3.3 µs typical and tens of microseconds unproven worst case for locked Linux. The bound does not depend on other tasks' behaviour, whereas on Linux a rogue task could still disturb it through cache and bus effects.

Memory available to the machine-learning task. An RTOS image is small (1 to 50 MB with its libraries) against about 300 MB for Linux with services, so roughly 250 MB more is left; but there is **no virtual memory**: the ML task needs a contiguous 1.5 GB region (or several pre-allocated regions), a fixed partitioning chosen at design time, no overcommit, no page cache and no demand loading of the model file (the 300 MB model is copied into RAM, rather than mapped, costing 300 MB of resident memory instead of an evictable one). Also, many ML frameworks (TensorFlow, PyTorch, ONNX Runtime) assume Linux; they have to be ported or replaced by an embedded runtime (TensorFlow Lite Micro does not support a 1.5 GB model). So the memory is higher in principle but the ML task's software options shrink.

Effort of adding new tasks. On Linux a new task is a process in a container with a memory cgroup: easy to add, with standard tools, languages and libraries and a large developer community; on an RTOS without virtual memory every task needs a statically planned memory budget, can corrupt other tasks if the MPU regions are misconfigured (no per-process address spaces on a basic RTOS), and needs cross-compilation and vendor-specific tools. As the vendor wants to add more tasks in the future, Linux with locked memory has the lower long-term effort. A *hybrid* is often best: a small RTOS core (or a Linux-RT partition with a hypervisor) for the hard real-time control, and Linux for the

ML and logging tasks on the same chip, communicating through shared memory.

Complexity summary. Locked Linux: control-task delay O(1) per access after pre-faulting but with no proved bound; memory O(sum of task sizes) with the kernel at about 300 MB; new task cost: low. RTOS without virtual memory: control-task delay O(1) with a proved bound; memory O(sum of fixed partitions) with a kernel at about 10 MB; new task cost: high.

7.3 Evidence summary for Q7

Requirements stated as measurable statements (Stage 1), a comparison table of the mechanisms with the use of `mlock` (Stage 2), a measured worst-case delay with and without locking (172 µs against 3.3 µs) and a replacement-simulator verification (33 % against 0 % faults for the RT task) (Stage 3), and a defence with the effect of a fourth 500 MB task and one remaining limitation (Stage 4).

8. Conclusion of Assignment 3

Memory management is a series of trade-offs that the measurements make concrete. Dynamic linking reduces the executable from 785,704 to 16,472 bytes and lets a security fix be deployed by replacing one file, at the cost of a 29 % slower start (Q1). Paging removes the external fragmentation that left 76 % of free memory unusable in the segment experiment, at the price of half a page of internal fragmentation per process (0.08 % with 4 KB pages) and of a page table, which the TLB and multi-level tables make affordable (EAT 111 ns at a 99 % hit ratio) (Q2, Q3). Demand paging lets a program larger than RAM run, but only if the fault rate stays below about 1 in 800,000 accesses; a rate of 1 in 1,000 makes it 81 times slower, which is why locality matters more than RAM size (Q4). LRU and its clock approximation halve the page faults of FIFO on workloads with locality (6,111 against 11,929 with 7 frames), and make no difference for random access; FIFO also shows Belady's anomaly (Q5). RAID 10 gives the best protection and write performance for the records server, with RAID 6 for the archive (Q6). And for the real-time edge device, locking memory removed all page faults of the control task and reduced its worst-case delay from 172 µs to 3.3 µs, with the limitation that only a real-time OS can give a provable bound (Q7).

Appendix A – Additional Source Listings

The programs below are part of the repository and were used for the calculations and experiments in this report.

A.1 libenc_v2.c (the library with the security fix, Q1)

Listing: libenc_v2.c

```
/* The bank's encryption library, version 2 (security fix: key changes per byte, for \
       demonstration only). */
#include <stddef.h>
int enc_version(void) { return 2; }
void encrypt(char *buf, size_t n) { for (size_t i = 0; i < n; i++) buf[i] ^= (char)( \
      0xA7 + i * 31); }
```

A.2 q2_edge.c (page-boundary tests, Q2)

Listing: q2_edge.c

```
/* Test addresses at the start and end of a page and beyond the last page (page size \
       1024, 8 pages). */
#include <stdio.h>
#define PS 1024
#define PAGES 8
int pt[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};
int main(void)
{
    unsigned t[] = {0, 1023, 1024, 2047, 2048, 7167, 8191, 8192, 8193};
    for (int i = 0; i < 9; i++) {
        unsigned p = t[i] / PS, d = t[i] % PS;
        printf("address %5u: page %u offset %4u -> ", t[i], p, d);
        if (p >= PAGES) puts("INVALID (beyond the last page: trap, process killed)") \
      ;
        else if (pt[p] < 0) puts("page fault (valid page, not in memory)");
        else printf("physical %u (frame %d)\n", pt[p] * PS + d, pt[p]);
    }
    return 0;
}
```

A.3 q2_calc.py (page-table size, EAT and internal fragmentation, Q2)

Listing: q2_calc.py

```
import random
print("== Page-table size (single level, 4-byte entries for 32-bit, 8-byte entries f \
      or 48-bit), page size 4 KB")
for bits, e in ((32, 4), (48, 8)):
    entries = 2 ** (bits - 12)
    print("  %d-bit address space: %s entries x %d bytes = %s" % (bits, format(entri \
      es, ','), e, "%.0f MB" % (entries * e / 2**20) if entries * e < 2**40 else "%. \
```

```
      0f GB" % (entries * e / 2**30)))
print("  (48-bit: 4-level table: only used parts exist; a 10 MB process needs about  \
      1 + 1 + 1 + 3 = 6 pages = 24 KB)")
print("\n== Effective memory-access time, memory 100 ns, TLB 10 ns, 1-level table")
def eat(h, mem=100, tlb=10, levels=1): return h * (tlb + mem) + (1 - h) * (tlb + lev \
      els * mem + mem)
print("  without TLB (1 level): %d ns; (4 levels): %d ns" % (2 * 100, 5 * 100))
for h in (0.80, 0.95, 0.99):
    print("  hit ratio %2d%%: 1 level %.1f ns  (slow-down %.0f%% over 100 ns); 4 lev \
      els %.1f ns" % (h * 100, eat(h), (eat(h) - 100), eat(h, levels=4)))
print("\n== Internal fragmentation for 100 processes with random sizes (seed 42, siz \
      es 10 KB - 5 MB; 2 MB pages use 1-64 MB processes)")
random.seed(42)
sizes = [random.randint(10 * 1024, 5 * 1024 * 1024) for _ in range(100)]
big = [random.randint(1 * 2**20, 64 * 2**20) for _ in range(100)]
for ps, lab, data in ((1024, "1 KB", sizes), (4096, "4 KB", sizes), (2 * 2**20, "2 M \
      B", big)):
    waste = sum((-s) % ps for s in data); total = sum(data)
    print("  page %5s: total memory requested %9.1f MB, internal fragmentation %10.1 \
      f KB (%.4f%%), average %.1f KB per process, pages needed %d" % (lab, total / 2 \
      **20, waste / 1024, 100 * waste / total, waste / 1024 / 100, sum(-(-s // ps) f \
      or s in data)))
print("  expected waste per process = page size / 2: 0.5 KB, 2 KB, 1024 KB")
```

A.4 q3_frag.py (external fragmentation of 20 segments, Q3)

Listing: q3_frag.py

```
"""Load and remove 20 segments of random sizes in 1024 KB of memory (first fit) and  \
      measure external fragmentation."""
import random
random.seed(7)
MEM = 1024
free = [(0, MEM)]            # list of (start, size) holes
placed = {}
def alloc(i, size):
    for k, (s, sz) in enumerate(free):
        if sz >= size:
            placed[i] = (s, size)
            if sz == size: free.pop(k)
            else: free[k] = (s + size, sz - size)
            return True
    return False
def release(i):
    s, sz = placed.pop(i); free.append((s, sz)); free.sort()
    merged = []
    for a, b in free:
        if merged and merged[-1][0] + merged[-1][1] == a: merged[-1] = (merged[-1][0 \
      ], merged[-1][1] + b)
        else: merged.append((a, b))
    free[:] = merged
sizes = [random.randint(20, 120) for _ in range(20)]
```

```
for i, s in enumerate(sizes): alloc(i, s)
print("20 segments loaded, sizes (KB):", sizes, " total =", sum(sizes), "KB")
for i in random.sample(range(20), 10): release(i)
tot = sum(s for _, s in free); big = max(s for _, s in free)
print("after removing 10 segments: free holes =", free)
print("  total free = %d KB, largest hole = %d KB, number of holes = %d" % (tot, big \
      , len(free)))
print("  external fragmentation = 1 - largest/total = %.1f %%" % (100 * (1 - big / t \
      ot)))
req = big + 1
print("  a new segment of %d KB (smaller than the %d KB free) can be placed: %s" % ( \
      req, tot, alloc(99, req)))
free.sort()
# compaction
print("  after compaction one hole of %d KB would exist (cost: copy %d KB of live se \
      gments)" % (tot, sum(s for _, s in placed.values())))
print("\nTable sizes for a 4 GB process using 4 segments: segment table = 4 entries  \
      x 8 bytes = 32 bytes")
print("Single-level page table (4 KB pages, 4 GB): 1,048,576 entries x 4 bytes = 4 M \
      B; two-level: 4 KB top + used 4 KB leaf tables")
```

A.5 q4_eat.py (effective access time and frame allocation, Q4)

Listing: q4_eat.py

```
mem, svc = 100e-9, 8e-3
print("== Effective access time EAT = (1-p) x 100 ns + p x 8 ms")
for lab, p in (("1 in 1,000", 1e-3), ("1 in 100,000", 1e-5), ("1 in 1,000,000", 1e-6 \
      )):
    eat = (1 - p) * mem + p * svc
    print("  p = %-15s EAT = %9.1f ns  (slow-down factor %.1f)" % (lab, eat * 1e9, e \
      at / mem))
p10 = 0.1 * mem / (svc - mem)
print("\n== Maximum page-fault rate for a 10%% slow-down: EAT <= 110 ns -> p <= 10 n \
      s / (8,000,000 ns - 100 ns) = %.3e  (about 1 fault in %d accesses)" % (p10, ro \
      und(1 / p10)))
print("\n== 24 GB program on 16 GB RAM: if the program touches memory with locality  \
      so that p = 1e-6, EAT = %.1f ns" % (((1 - 1e-6) * 100e-9 + 1e-6 * 8e-3) * 1e9) \
      )
print("== 32 GB machine, no faults after loading: EAT = 100 ns; with p=1e-6 on the s \
      mall machine EAT = 108 ns (8% slower)")
print("\n== Equal and proportional frame allocation: 62 frames, processes of 10 and  \
      127 pages")
print("  equal: 31 and 31 frames; proportional: 62 x 10/137 = %.1f -> 4, and 62 x 12 \
      7/137 = %.1f -> 57 frames" % (62 * 10 / 137, 62 * 127 / 137))
```

A.6 q5_trace.py (step trace for the hand verification, Q5)

Listing: q5_trace.py

```
"""Step-by-step trace used for the hand verification (3 frames, textbook string)."""
ref = [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1]
def trace(policy):
    frames = []; faults = 0; rows = []
    for i, p in enumerate(ref):
        hit = p in frames
        if hit:
            if policy == 'LRU': frames.remove(p); frames.append(p)
        else:
            faults += 1
            if len(frames) == 3:
                if policy == 'OPT':
                    def nu(x):
                        try: return ref.index(x, i + 1)
                        except ValueError: return 10**9
                    frames.remove(max(frames, key=nu))
                else: frames.pop(0)
            frames.append(p)
        rows.append((p, sorted(frames) if policy == 'OPT' else list(frames), 'hit' i \
      f hit else 'FAULT'))
    return rows, faults
for pol in ('FIFO', 'LRU', 'OPT'):
    rows, f = trace(pol)
    print("%s, 3 frames: %d faults" % (pol, f))
    print("  ref : " + " ".join("%2d" % r[0] for r in rows))
    print("  flt : " + " ".join(" X" if r[2] == 'FAULT' else " ." for r in rows))
    print()
```

A.7 q6_raid.py (RAID capacity and data-loss probabilities, Q6)

Listing: q6_raid.py

```
import math
n, tb = 6, 4
print("Usable capacity of 6 disks x 4 TB")
for name, cap, tol, rd, wr in (("RAID 0", n * tb, "0", "6x", "6x"), ("RAID 1 (3 mirr \
      ored pairs)", n * tb // 2, "1 (up to 3, one per pair)", "up to 6x", "3x"), ("R \
      AID 5", (n - 1) * tb, "1", "5x", "about 1.5x (4 I/Os per small write)"),
                                ("RAID 6", (n - 2) * tb, "2", "4x", "about 1x (6 I \
      /Os per small write)"), ("RAID 10 (3 mirrored pairs striped)", n * tb // 2, "1 \
       per pair (at least 1, up to 3)", "6x", "3x")):
    print("  %-36s usable %2d TB (%.0f%%)  survives %-30s read %s  write %s" % (name \
      , cap, 100 * cap / (n * tb), tol, rd, wr))
print("\nProbability of data loss during one rebuild (a disk has failed; annual fail \
      ure rate AFR = 3%, rebuild takes 24 h)")
afr = 0.03; hours = 24; p = afr * hours / 8760     # probability that one given disk \
       fails during the rebuild window
print("  p(one given surviving disk fails during rebuild) = %.2e" % p)
print("  RAID 1 (pair): the single partner disk must fail:           P = %.2e" % p)
print("  RAID 5 (6 disks): any of the 5 survivors fails:             P = 1-(1-p)^5 = \
```

```
       %.2e   (about %.1fx RAID 1)" % (1 - (1 - p) ** 5, (1 - (1 - p) ** 5) / p))
print("  RAID 6 (6 disks): two more of the 5 survivors must fail:    P = C(5,2) p^2  \
      = %.2e" % (10 * p * p))
print("  RAID 10 (3 pairs): only the failed disk's partner matters:  P = %.2e" % p)
ure = 1e-14   # unrecoverable read error per bit
bits = 5 * 4e12 * 8
print("  Rebuild of RAID 5 reads 5 x 4 TB = %.0e bits; with URE rate 1e-14 per bit e \
      xpected read errors = %.2f -> P(at least one) = %.0f%%" % (bits, bits * ure, 1 \
      00 * (1 - math.exp(-bits * ure))))
print("  RAID 1/10 rebuild reads only 4 TB = %.1e bits: P(URE) = %.0f%%" % (4e12 * 8 \
      , 100 * (1 - math.exp(-4e12 * 8 * ure))))
print("\nSwap sizing example: 64 GB RAM records server, hibernation not used: swap = \
       8-16 GB (safety margin for peaks) on SSD; swappiness 10")
```

A.8 sys.sh (system views used in Q6)

Listing: sys.sh

```
run() { echo "\$ $*"; eval "$*" 2>&1 | head -${LINES_MAX:-14}; }
run "lsblk"
run "fdisk -l"
run "cat /proc/swaps; swapon --show"
run "free -h"
run "df -hT / | head -3"
run "cat /sys/block/vda/queue/scheduler 2>/dev/null || ls /sys/block"
```

Appendix B – README.md Template for the GitHub Repository

```
# CSA04 - Operating Systems - Assignment 3 (CO3)
Name: <your name>            Register number: <register number>
Compiler: gcc 13.3.0         OS: Ubuntu 24.04 LTS (Linux 6.x)
Compile (C):  gcc -std=c11 -Wall -Wextra -pthread -O2 file.c -o file     (use -std=g \
      nu11 for mmap/dlopen programs, add -ldl if needed)

| Folder | Program | How to run                   | Result (one line)                \
                  |
|--------|---------|------------------------------|--------------------------------- \
      -------------|
| Q1     | app.c   | bash demo.sh                 | dynamic 16,472 B vs static 785,7 \
      04 B         |
| Q2     | q2_page.c | ./q2_page                  | matches the expected output      \
                   |
| Q3     | q3_seg.c  | ./q3_seg                   | 8 translations, 4 traps          \
                   |
| Q4     | q4_cow.c  | ./q4_cow                   | child shares 103 MB, copies only \
       what it writes |
| Q5     | q5_replace.c | ./q5_replace            | FIFO 15/10, LRU 12/8, OPT 9/8; B \
      elady 9->10 |
| Q6     | q6_raid.py | python3 q6_raid.py        | RAID 10 recommended              \
```

```
                  |
| Q7     | q7_mlock.c | ./q7_mlock                | worst case 172 us -> 3.3 us with \
       mlock       |

Commits: one per question (git log shows the history).
```

Appendix C – Viva Preparation: Questions and Short Answers

The assignment states that any part of the code or the design may be questioned. The following short answers cover the points most likely to be asked.

1. **Why is the first read in the copy loop counted even though it returns 0?** Because the program counts every call to `read()`, and the call that discovers end of file is also a system call.
2. **What is the difference between static and dynamic linking, and which one allowed the library to be replaced without recompiling?** Static linking copies the library into the executable; dynamic linking resolves the library at load time, so replacing the `.so` file changes the behaviour of all dynamically linked programs.
3. **Why does the page table lookup need a TLB?** Without it each access costs two memory references (or five with four levels); the TLB caches translations so that at a 99 % hit ratio the access time is 111 ns instead of 200 ns.
4. **What is the difference between a page fault and a segmentation fault?** A page fault is a valid reference to a page that is not in memory (the OS loads it and continues); a segmentation fault is an illegal reference (unmapped address or permission violation), for which the OS delivers SIGSEGV.
5. **Why is the maximum acceptable fault rate about 1 in 800,000?** From EAT = (1 - p) 100 + p 8,000,000 <= 110 ns, so p <= 10 / 7,999,900 = 1.25 × 10^-6.
6. **Explain Belady's anomaly.** FIFO with 4 frames faults 10 times and with 3 frames 9 times on the string 1 2 3 4 1 2 5 1 2 3 4 5, because FIFO is not a stack algorithm.
7. **Why can LRU not suffer from the anomaly?** The set of pages in memory with n frames is a subset of the set with n + 1 frames (stack property).
8. **Why does mlock help a real-time task?** It pins the pages in RAM, so no page fault (minor or major) can occur in the critical loop; measured worst case fell from 172 µs to 3.3 µs.
9. **Why is RAID 10 better than RAID 5 for a database?** A small write needs 2 disk writes instead of 4 I/Os, the rebuild copies one disk (27 % URE probability instead of 80 %), and the data-loss probability in the rebuild window is five times lower.
10. **What would change if the page size were 2 MB?** Internal fragmentation grows to 1 MB per process on average (3.2 % in the experiment), the page table for 1 GB shrinks

from 1 MB to 2 KB and the TLB coverage grows 512 times.

Appendix D – Worked Examples for the Viva

D.1 Address translation with a page table (page size 1,024, table {5, 2, -1, 7, 0, -1, 3, -1})

To translate a logical address a: p = a div 1,024 and d = a mod 1,024; if p is not less than 8 the address is invalid; if the table entry is -1 a page fault occurs; otherwise the physical address is frame x 1,024 + d.

Logical address	Page p	Offset d	Table entry	Result
0	0	0	5	5,120
1023	0	1023	5	6,143 (last byte of page 0)
1024	1	0	2	2,048 (first byte of page 1)
1500	1	476	2	2,524
2049	2	1	-1	page fault
3100	3	28	7	7,196
4096	4	0	0	0
5500	5	380	-1	page fault
6200	6	56	3	3,128
7167	6	1023	3	4,095
8192	8	0	none	invalid address
9000	8	808	none	invalid address

D.2 Effective access time with a TLB (memory 100 ns, TLB 10 ns)

EAT = h (10 + 100) + (1 - h) (10 + 100 + 100) for a one-level table: h = 0.80 gives 0.8 x 110 + 0.2 x 210 = 130 ns; h = 0.95 gives 104.5 + 10.5 = 115 ns; h = 0.99 gives 108.9 + 2.1 = 111 ns. With four levels a miss costs 10 + 400 + 100 = 510 ns: h = 0.99 gives 108.9 + 5.1 = 114 ns.

D.3 Page-fault rate for a 10 % slow-down

0.1 x 100 = (8,000,000 - 100) p, so p = 10 / 7,999,900 = 1.25 x 10^-6, one fault in 800,000 accesses. For one fault in 1,000 the EAT is 99.9 + 8,000 = 8,099.9 ns.

D.4 LRU trace on the textbook string with 3 frames (most recent last)

Reference	Frames after the reference	Fault?
7	[7]	yes
0	[7,0]	yes
1	[7,0,1]	yes
2	[0,1,2]	yes (7 was least recently used)
0	[1,2,0]	no
3	[2,0,3]	yes (1 evicted)
0	[2,3,0]	no
4	[3,0,4]	yes (2 evicted)
2	[0,4,2]	yes (3 evicted)
3	[4,2,3]	yes (0 evicted)
0	[2,3,0]	yes (4 evicted)
3	[2,0,3]	no
2	[0,3,2]	no
1	[3,2,1]	yes (0 evicted)
2	[3,1,2]	no
0	[1,2,0]	yes (3 evicted)
1	[2,0,1]	no
7	[0,1,7]	yes (2 evicted)
0	[1,7,0]	no
1	[7,0,1]	no

The faults are counted as 12, as printed by the program.

D.5 Capacity of RAID 6 and RAID 10 with six 4 TB disks

RAID 6: (6 - 2) x 4 TB = 16 TB. RAID 10: 6 / 2 x 4 TB = 12 TB. RAID 5: (6 - 1) x 4 TB = 20 TB. RAID 0: 24 TB. RAID 1 (three pairs): 12 TB.

Appendix E – Test Log

Program	Command	Result
libenc.so, app_dynamic, app_static	bash q1/demo.sh	16,472 B against 785,704 B; version 2 picked up without recompiling by the dynamic program only
app_dl.c	./app_dl encrypt (library missing)	dlopen fails, fallback message, exit status 2
q2_page.c	compile and run (page size 1024 and 2048)	identical to the expected output; 2048 matches the hand table
q2_faults.c	./q2_faults	25,600 faults for 25,600 pages, none on the second pass
q3_seg.c	./q3_seg	8 translations: 3 valid, 5 traps
q3_segv.c	./q3_segv and ./q3_segv ro	SEGV_MAPERR and SEGV_ACCERR, exit status 139
q4_cow.c	./q4_cow	12,800 copy-on-write faults for 50 MB written
q5_replace.c	compile and run	identical to the expected output
q5_exp.c	./q5_exp	FIFO twice the faults of LRU for locality workloads
q6_raid.py	python3 q6_raid.py	capacities and loss probabilities
q7_mlock.c	./q7_mlock	worst case 172 us without locking, 3.3 us with mlock
q7_sim.py	python3 q7_sim.py	RT task 33 % faults unlocked, 0 % locked

Appendix F – Glossary

Term	Meaning
Address binding	Mapping of symbolic names to memory addresses at compile, load or execution time
Position-independent code	Code that works at any load address, so that shared libraries can be mapped anywhere
Page / frame	Fixed-size block of logical memory / of physical memory
TLB	Small cache of recent page-table entries
Internal fragmentation	Unused space inside an allocated block (last page of a process)

Term	Meaning
External fragmentation	Free memory that is split into pieces too small to use
Segment	Variable-size logical unit (code, data, stack) with base and limit
Demand paging	Loading a page only when it is referenced
Copy-on-write	Sharing pages until one process writes, then copying that page
Thrashing	Excessive paging so that little useful work is done
Working set	Pages used in the most recent window of references
Belady's anomaly	More frames giving more faults under FIFO
Stack algorithm	Replacement algorithm whose resident set with n frames is a subset of that with n + 1 frames
RAID	Redundant array of independent disks
mlock	System call that pins pages in RAM so that they are never paged out
URE	Unrecoverable read error of a disk

Appendix G – Extended Calculations

Listing: ext_calc.py

```
"""Appendix G: extended calculations used in the discussion (page tables, EAT sweep, \
       loss probabilities, swap sizing)."""
print("== Page-table size for different page sizes (single level, 4-byte entries, 32 \
      -bit address space = 4 GB)")
print("%10s %14s %16s %18s" % ("page size", "pages", "table size", "offset bits"))
for ps, nm in ((512, "512 B"), (1024, "1 KB"), (4096, "4 KB"), (16384, "16 KB"), (65 \
      536, "64 KB"), (2 << 20, "2 MB"), (4 << 20, "4 MB")):
    n = 2 ** 32 // ps; print("%10s %14s %16s %18d" % (nm, format(n, ","), "%.1f KB"  \
      % (n * 4 / 1024) if n * 4 < 2 ** 20 else "%.1f MB" % (n * 4 / 2 ** 20), ps.bit \
      _length() - 1))
print("\n== Effective access time (ns) against TLB hit ratio, memory 100 ns, TLB 10  \
      ns")
print("%8s %12s %12s %12s" % ("hit", "1 level", "2 levels", "4 levels"))
for h in (0.50, 0.60, 0.70, 0.80, 0.90, 0.95, 0.98, 0.99, 0.995, 0.999):
    f = lambda L: h * 110 + (1 - h) * (10 + L * 100 + 100)
    print("%8.3f %12.1f %12.1f %12.1f" % (h, f(1), f(2), f(4)))
print("\n== Demand paging: EAT (ns) for fault rate p and service time 8 ms (hard dis \
      k), 100 us (SSD)")
print("%12s %14s %14s" % ("p", "hard disk", "SSD"))
for p in (1e-2, 1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8):
    print("%12.0e %14.1f %14.1f" % (p, (1 - p) * 100 + p * 8e6, (1 - p) * 100 + p *  \
      1e5))
print("\n== Maximum fault rate for a 10%% slow-down: hard disk %.2e, SSD (100 us) %. \
      2e" % (10 / (8e6 - 100), 10 / (1e5 - 100)))
print("\n== Probability of losing a RAID 5 / RAID 6 / RAID 10 set during a 24-hour r \
      ebuild for different annual failure rates (6 disks)")
```

```
print("%6s %14s %14s %14s" % ("AFR", "RAID 5", "RAID 6", "RAID 10"))
for afr in (0.01, 0.02, 0.03, 0.05, 0.08):
    p = afr * 24 / 8760
    print("%5.0f%% %14.2e %14.2e %14.2e" % (afr * 100, 1 - (1 - p) ** 5, 10 * p * p, \
       p))
print("\n== Swap sizing rules of thumb (no hibernation)")
for ram in (2, 4, 8, 16, 64):
    print("  RAM %3d GB: swap %5.1f - %5.1f GB" % (ram, ram * (1.0 if ram <= 2 else  \
      0.5 if ram <= 8 else 0.25 if ram <= 16 else 0.15), ram * (2.0 if ram <= 2 else \
       1.0 if ram <= 8 else 0.5 if ram <= 16 else 0.25)))
print("\n== Number of page-table pages for one process mapping m MB contiguously (x8 \
      6-64, 4 KB pages, 512 entries per table)")
for mb in (1, 10, 100, 1024, 24 * 1024):
    leaf = -(-mb * 256 // 512); l2 = -(-leaf // 512); l3 = -(-l2 // 512); total = le \
      af + l2 + l3 + 1
    print("  %6d MB: leaf tables %6d, level-2 %4d, level-3 %2d, root 1 -> %6d pages  \
      = %8.1f KB (%.3f%% of the mapped memory)" % (mb, leaf, l2, l3, total, total *  \
      4, 100 * total * 4 / (mb * 1024)))
```

$ cd q2 && python3 ext_calc.py

```
== Page-table size for different page sizes (single level, 4-byte entries, 32-bit ad \
      dress space = 4 GB)
 page size          pages       table size        offset bits
     512 B      8,388,608          32.0 MB                  9
      1 KB      4,194,304          16.0 MB                 10
      4 KB      1,048,576           4.0 MB                 12
     16 KB        262,144           1.0 MB                 14
     64 KB         65,536         256.0 KB                 16
      2 MB          2,048           8.0 KB                 21
      4 MB          1,024           4.0 KB                 22

== Effective access time (ns) against TLB hit ratio, memory 100 ns, TLB 10 ns
     hit      1 level     2 levels     4 levels
   0.500        160.0        210.0        310.0
   0.600        150.0        190.0        270.0
   0.700        140.0        170.0        230.0
   0.800        130.0        150.0        190.0
   0.900        120.0        130.0        150.0
   0.950        115.0        120.0        130.0
   0.980        112.0        114.0        118.0
   0.990        111.0        112.0        114.0
   0.995        110.5        111.0        112.0
   0.999        110.1        110.2        110.4

== Demand paging: EAT (ns) for fault rate p and service time 8 ms (hard disk), 100 u \
      s (SSD)
           p      hard disk            SSD
       1e-02        80099.0         1099.0
       1e-03         8099.9          199.9
       1e-04          900.0          110.0
       1e-05          180.0          101.0
```

```
       1e-06          108.0          100.1
       1e-07          100.8          100.0
       1e-08          100.1          100.0

== Maximum fault rate for a 10% slow-down: hard disk 1.25e-06, SSD (100 us) 1.00e-04

== Probability of losing a RAID 5 / RAID 6 / RAID 10 set during a 24-hour rebuild fo \
      r different annual failure rates (6 disks)
   AFR         RAID 5         RAID 6        RAID 10
    1%       1.37e-04       7.51e-09       2.74e-05
    2%       2.74e-04       3.00e-08       5.48e-05
    3%       4.11e-04       6.76e-08       8.22e-05
    5%       6.85e-04       1.88e-07       1.37e-04
    8%       1.10e-03       4.80e-07       2.19e-04

== Swap sizing rules of thumb (no hibernation)
  RAM   2 GB: swap   2.0 -   4.0 GB
  RAM   4 GB: swap   2.0 -   4.0 GB
  RAM   8 GB: swap   4.0 -   8.0 GB
  RAM  16 GB: swap   4.0 -   8.0 GB
  RAM  64 GB: swap   9.6 -  16.0 GB

== Number of page-table pages for one process mapping m MB contiguously (x86-64, 4 K \
      B pages, 512 entries per table)
       1 MB: leaf tables      1, level-2    1, level-3  1, root 1 ->      4 pages =  \
          16.0 KB (1.562% of the mapped memory)
      10 MB: leaf tables      5, level-2    1, level-3  1, root 1 ->      8 pages =  \
          32.0 KB (0.312% of the mapped memory)
     100 MB: leaf tables     50, level-2    1, level-3  1, root 1 ->     53 pages =  \
         212.0 KB (0.207% of the mapped memory)
    1024 MB: leaf tables    512, level-2    1, level-3  1, root 1 ->    515 pages =  \
        2060.0 KB (0.196% of the mapped memory)
   24576 MB: leaf tables  12288, level-2   24, level-3  1, root 1 ->  12314 pages =  \
       49256.0 KB (0.196% of the mapped memory)
```

These tables support the page-size choice in Q2 (4 KB gives a 4 MB single-level table for 32 bits, 2 MB pages shrink it to 8 KB), the TLB analysis (a 99 % hit ratio gives 111 ns for one level and 114 ns for four), the demand-paging limit in Q4 (an SSD with 100 microsecond service time tolerates a fault rate 80 times higher than a hard disk: 1.0 x 10^-4 against 1.25 x 10^-6), the RAID loss probabilities in Q6 for several failure rates, and the page-table overhead of a process mapping 24 GB (about 0.2 % of the mapped memory).